

ne unos accesos centrales en una situación en que los objetos de la clase pueden estar distribuidos en muchas máquinas. En lugar de utilizar las clases como si fuesen objetos con estado, es mejor introducir objetos explícitos para almacenar en ellos cualquier información compartida que pudiera precisarse. Tanto el modelo como los costes se vuelven más evidentes.

Los constructores (operación de creación, operaciones de fábrica) tienen necesariamente un alcance de código del nivel de clase, porque (todavía) no hay ninguna instancia sobre la que puedan operar. Esto es una aplicación correcta y necesaria del alcance de clase. Las otras clases de operaciones cuyo alcance original es del nivel de clase tienen las mismas dificultades que los atributos, a saber, implican una información global centralizada relativa a las instancias de las clases, lo cual no resulta práctico en sistemas distribuidos.

El alcance de destino tiene una utilidad limitada y sólo debería utilizarse en circunstancias especiales generalmente, sólo para propósitos detallados de programación.

alcance de destino

Especificación de si un valor es una instancia o un clasificador.

Véase alcance.

Discusión

El alcance de destino se usa sobre todo para almacenar clases como valores de atributos o como destinos de asociaciones. Tiene una utilidad limitada. La palabra alcance sin aditamentos significa alcance de propietario.

alcance de propietario

Se trata de una indicación de si la característica es aplicable a un objeto individual o si es compartida por toda una clase.

Véase también alcance, alcance de destino.

Semántica

El alcance de propietario indica si existe o no una ranura de atributo distinta para cada instancia de una clase, o si existe una única ranura para toda la clase. Para un operador, el alcance de propietario indica si una operación se aplica a una operación o bien a la clase en sí (tal como sucede en un operador de creación). En algunas ocasiones se denomina simplemente alcance. Los valores posibles son:

instancia

Cada instancia de clasificador posee su propia copia exclusiva de una ranura de atributo. Los valores de una ranura son independientes de los valores de otras ranuras. Ésta es la situación normal.

Para un operador, el operador es aplicable a un objeto individual.

clase

El clasificador en sí posee una copia de la ranura de atributo. Todas las instancias del clasificador comparten el acceso a la única ranura. Si el lenguaje permite clases como objetos reales entonces esto es un atributo de la clase en sí como objeto.

Para un operador, el operador se aplica a toda la clase, como sucede en un operador de creación o en un operador que proporcione estadísticas relativas a todo un conjunto de instancias.

Notación

Los atributos u operadores con alcance de clase se subrayan (Figura 13.14). Los atributos u operadores con alcance de instancia no se subrayan.

Reserva
fecha: Fecha
<u>avanceMax: Hora</u>
<u>crear(fecha: Fecha)</u>
destruir()

Figura 13.14 Atributo y operación con alcance de clase

Discusión

Para una asociación, se diría si la posición de origen de un enlace contiene instancias o clasificadores. Pero esta información se puede especificar como el alcance de destino en la otra dirección, así que el alcance de propietario es innecesario y, por tanto, no se usa para las asociaciones.

alcance original

Indicador de si una ranura es propiedad de una instancia o de una clase.

Véase alcance.

amigo/a

Una dependencia de uso que concede permiso al cliente para tener acceso al proveedor, incluso aunque el cliente no tenga la suficiente visibilidad para tener acceso al proveedor.

Véase también acceso, importar, visibilidad.

Semántica

La dependencia amigo se utiliza para conceder permiso a una clase u operación para usar el contenido de una clase, aunque el permiso fuera insuficiente. Es una excepción explícita a las

reglas normales de permiso entre los elementos afectados. Esta capacidad se debe utilizar cuidadosa y económicamente.

Notación

La dependencia amigo se representa como una flecha de línea discontinua desde la operación o la clase que adquiere el permiso a la clase cuyo contenido se hace disponible; la palabra clave de estereotipo «**friend**» se une a la flecha.

análisis

Etapas de un sistema que captura los requisitos y el dominio del problema. El análisis se centra en lo que hay que hacer, mientras que el diseño se centra en cómo hacerlo. En un proceso iterativo, estas etapas no tienen por qué realizarse de forma secuencial. El resultado de esta etapa se representa mediante modelos del nivel de análisis, especialmente la vista de casos de uso y la vista estática. Contrastar análisis con diseño, implementación y despliegue.

Véase fases de modelado, proceso de desarrollo.

antecesor

Elemento que se encuentra siguiendo una ruta en una o más relaciones padre.

Véase generalización, padre.

argumento

Valor específico de un parámetro.

Véase también ligadura, parámetro, principio de capacidad de sustitución.

Semántica

Una instancia de un mensaje en tiempo de ejecución tiene una lista de valores de los argumentos, cada uno de los cuales es un valor cuyo tipo debe ser consistente con el tipo declarado por el parámetro correspondiente en la declaración de la operación o señal. Un valor es consistente si su clase o tipo de dato es la misma o un descendiente de la declarada como tipo del parámetro. Según el principio de sustitución, se puede utilizar el valor de un descendiente en cualquier sitio donde se haya declarado un antecesor como tipo del parámetro. La implementación de un valor depende del simulador o entorno de ejecución en el que aparece.

En una colaboración o en una máquina de estados pueden aparecer expresiones que impliquen acciones. Dentro de estas expresiones, las llamadas y los envíos de mensajes requieren especificaciones de argumentos, que son también expresiones. Cuando se evalúan esas expresiones en tiempo de ejecución, deben evaluarse utilizando valores consistentes con los parámetros declarados con los que se corresponden.

Sin embargo, al realizar la ligadura de una plantilla, los argumentos aparecen dentro de un modelo UML durante el propio modelado. En estos casos, los argumentos se representan mediante expresiones en algún lenguaje, normalmente un lenguaje de restricciones o de programación. Los argumentos de las plantillas pueden incluir no sólo valores dato ordinarios u objetos, sino también clasificadores. En este último caso, el tipo de parámetro correspondiente deberá ser **Classifier** (clasificador) o cualquier otro metatipo. El valor de un argumento de plantilla debe fijarse en tiempo de modelado, y no es posible utilizarlo para representar un argumento en tiempo real. No utilice plantillas si no se han fijado los parámetros en tiempo de modelado.

argumento formal

Véase parámetro.

arquitectura

Estructura organizativa de un sistema que incluye su descomposición en partes, conectividad, mecanismos de interacción y principios de guía que proporcionan información sobre el diseño del mismo.

Véase paquete.

Semántica

La arquitectura es el conjunto de decisiones significativas sobre la organización de un sistema software. Incluye la selección de elementos estructurales y las interfaces mediante las que se conectan, la organización a gran escala de los elementos estructurales y la topología de su conexión, su comportamiento en las colaboraciones entre dichos elementos, los mecanismos importantes de que se dispone en el sistema y el estilo arquitectónico que guía su organización. Por ejemplo, la decisión de construir un sistema en dos capas, cada una de las cuales contiene un pequeño número de subsistemas que se comunican de una forma particular es una decisión arquitectónica. La arquitectura de un software no está relacionada sólo con la estructura y el comportamiento, sino también con el uso, la funcionalidad, el rendimiento, la flexibilidad, la reutilización, la facilidad de comprensión, las restricciones y compromisos económicos y tecnológicos y con la estética.

Discusión

Las decisiones arquitectónicas sobre la descomposición de un sistema en partes se pueden capturar utilizando modelos, subsistemas, paquetes, y componentes. Las dependencias entre estos elementos son indicadores clave de la flexibilidad de la arquitectura y de la dificultad de modificar el sistema en el futuro.

Otra parte importante de una arquitectura son los mecanismos que proporciona para construir sobre ella, que deben ser capturados mediante patrones y colaboraciones.

Las decisiones no estructurales se pueden capturar utilizando valores etiquetados.

artefacto

Pieza de información utilizada o producida por un proceso de desarrollo de software, como un documento externo o el producto de un trabajo. Un artefacto puede ser un modelo, una descripción o un software.

asociación

Relación semántica entre dos o más clasificadores que implica la conexión entre sus instancias.

Véase también asociación binaria, asociación *n*-aria, clase asociación, extremo de asociación, generalización de asociaciones.

Semántica

Una asociación es una relación entre dos o más clasificadores que describe conexiones entre sus instancias. Los clasificadores que participan tienen posiciones ordenadas dentro de la asociación. La misma clase puede aparecer en más de una posición dentro de la asociación. Cada instancia de una asociación (denominada enlace) es una tupla (lista ordenada) de referencias a objetos, ya que la asociación es un conjunto de enlaces. Un objeto dado puede aparecer más de una vez en el conjunto de enlaces, o incluso más de una vez (en diferentes posiciones) en el mismo enlace si lo permite la definición de la asociación. Las asociaciones son el “pegamento” que mantiene unido un sistema, pues sin ellas sólo sería un conjunto de clases inconexas.

Estructura

Una asociación tiene un nombre opcional, pero la mayoría de sus descripciones se encuentran en una lista de extremos de asociación, cada uno de los cuales describe cómo participan los objetos de una determinada clase en la asociación. Téngase en cuenta que un extremo de una asociación es simplemente una parte de la descripción de una asociación, y no un concepto semántico o de notación separado.

Nombre. Una asociación puede tener opcionalmente un nombre: una cadena única con respecto a las clases y asociaciones del paquete que la contiene. Una clase asociación es tanto una asociación como una clase, pero sin embargo, las clases y las asociaciones comparten un único espacio de nombres. No es necesario que una asociación tenga un nombre, pues los nombres de roles de sus extremos proporcionan una forma alternativa de distinguir las asociaciones múltiples entre las mismas clases. Por convención, el nombre se lee en el orden en que las clases participantes aparecen en la lista: una Persona trabaja para una Empresa; un Vendedor vende un Coche a un Cliente.

Extremos de asociación. Una asociación contiene una lista ordenada de dos o más extremos de asociación. Por ordenado se entiende que los extremos pueden distinguirse y no son intercambiables. Cada extremo de una asociación define la participación de una clase en una posición dada (rol) en la asociación. La misma clase puede aparecer en más de una posición,

pero las posiciones no son intercambiables, en general. Cada extremo de asociación especifica propiedades que se aplican a los objetos que participan, como cuántas veces puede aparecer en la asociación un objeto simple (multiplicidad). Ciertas propiedades, como la posibilidad de navegación, se aplican sólo a las asociaciones binarias, pero la mayoría se aplican tanto a las binarias como a las n -arias.

Véase extremo de asociación para más detalles.

Instanciación

Un enlace es una instancia de una asociación, que contiene una ranura para cada extremo de la asociación. Cada ranura contiene una referencia a un objeto que es una instancia (directa o indirecta) de la clase especificada como clase del correspondiente extremo de la asociación. Un enlace no tiene identidad aparte de la lista de objetos que contiene. Los enlaces que forman una asociación forman un conjunto donde no pueden existir duplicados. El número de veces que aparece un objeto en un conjunto de enlaces debe ser compatible con la multiplicidad de cada extremo de la asociación. Por ejemplo, si la asociación **EntradasVendidas** conecta muchas entradas con un espectáculo, entonces cada entrada puede aparecer sólo una vez en un enlace, pero cada espectáculo puede aparecer muchas veces, cada una con una entrada diferente.

Se pueden crear y destruir enlaces según avance la ejecución de un sistema, ateniéndose a las restricciones de intercambio de los extremos de asociación. En algunos casos se puede crear o cambiar un enlace de un objeto de uno de los extremos de la asociación pero no del otro extremo. Un enlace se crea a partir de una lista de referencias. No tiene identidad por sí mismo, por lo que carece de sentido hablar de modificar su valor, aunque puede destruirse y crearse otro en su lugar. Un enlace de una clase asociación tiene uno o más valores de atributos además de la lista de objetos que define su identidad, y las operaciones pueden modificar los valores de los atributos mientras preservan las referencias a los objetos participantes.

Notación

Una asociación binaria se representa mediante una ruta continua que conecta los bordes de dos clases (véase Figura 13.15). Una asociación n -aria se representa mediante un rombo conectado

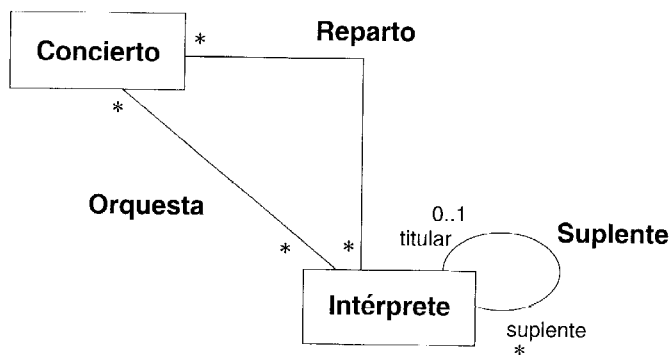


Figura 13.15 Asociaciones

a sus clases participantes mediante rutas (véase Figura 13.19). En la asociación binaria, el rombo se suprime por ser superfluo. Puede haber más de un final de ruta conectada a una misma clase.

Una ruta está formada por uno o más segmentos dibujados con línea continua conectados, normalmente segmentos de recta, aunque están permitidos los arcos y otras curvas, especialmente a la hora de mostrar una auto-asociación (una asociación en la que una clase aparece más de una vez). Los segmentos individuales no tienen semántica significativa. El usuario puede escoger el estilo de línea a utilizar.

Véase ruta.

Los extremos de las rutas tienen adornos que describen la participación de una clase en la asociación. Algunos adornos se representan en el extremo de la ruta, entre el segmento de línea y la caja de la clase. Si hay múltiples adornos, se sitúan secuencialmente comenzando por el extremo de la línea hasta el símbolo de la clase según el orden: flecha de navegación, rombo de agregación/composición y calificador (véase Figura 13.16).

Otros adornos, como las etiquetas de nombre, se sitúan cerca del elemento al que identifican. Los nombres de rol se sitúan cerca del final de la ruta.

Véase extremo de asociación para más detalles sobre la notación de los adornos.

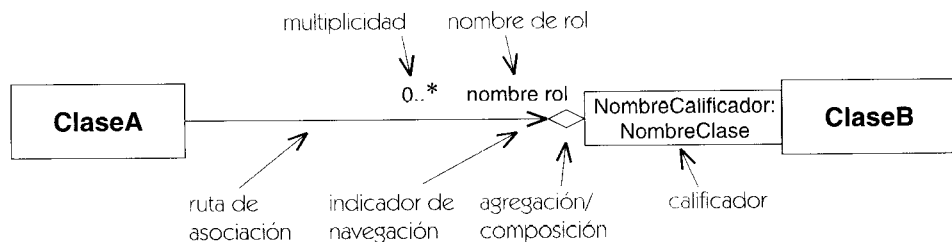


Figura 13.16 Orden de los adornos en un extremo de asociación

Nombre de asociación

El nombre de una asociación se sitúa cerca de la línea de la ruta, pero lo suficientemente lejos del extremo como para que no haya peligro de confusión. El peligro de confusión es meramente visual para las personas, ya que dentro de una herramienta gráfica no existe peligro de ambigüedad en la representación interna, por lo que la herramienta se encarga de determinar la distancia óptima. El nombre de la asociación se puede arrastrar de unos segmentos a otros de una asociación con múltiples segmentos sin que esto tenga ningún significado especial. El nombre de la asociación puede tener al lado un pequeño triángulo relleno que muestre el orden de las clases en la lista. Intuitivamente, la flecha del nombre muestra cómo leer el nombre. En la Figura 13.17, la asociación **TrabajaPara** entre la clase **Persona** y la clase **Empresa** debería tener el triángulo apuntando de **Persona** a **Empresa** y leerse “Persona trabaja para Empresa”. Obsérvese que poner el triángulo en el nombre es un aspecto únicamente de notación que indica el orden en que finaliza la asociación. En el propio modelo, los extremos están inherentemente ordenados, por lo que el nombre del modelo no tiene ni necesita una propiedad de ordenación.

Un estereotipo en la asociación se indica mostrando el nombre del estereotipo entre comillas (« ») enfrente o en lugar del nombre de la asociación. Se puede situar una lista de propiedades después o debajo del nombre de la asociación.

Ejemplo



Figura 13.17 Nombre de asociación

Clase asociación

Una clase asociación se representa asociando un símbolo de clase a la ruta de asociación mediante una línea discontinua. En asociaciones n -arias, la línea discontinua se conecta con el rombo de la asociación. Las propiedades “de clase” de la asociación se muestran en el símbolo de clase, mientras que las propiedades “de asociación” de la misma se muestran en la ruta. Observe sin embargo, que la estructura de modelado subyacente es un único elemento, incluso aunque para representarla se utilicen dos estructuras diferentes.

Véase clase asociación para más detalles.

Restricción Xor

La restricción **{xor}** conecta dos o más asociaciones que están conectadas a una misma clase simple (la clase base) en un extremo. Una instancia de la clase base puede participar solamente en una de las asociaciones conectadas por la restricción. Debe respetarse la multiplicidad de la asociación elegida. Si la multiplicidad de alguna asociación incluye la cardinalidad 0, una instancia de la clase base podría no tener enlaces desde la asociación; si no, debería tener uno.

Una restricción **xor** se representa mediante una línea discontinua etiquetada con la cadena de restricción **{xor}** que conecta dos o más asociaciones que tienen una clase en común (Figura 13.18). Los nombres de roles de los extremos de la asociación más lejanos a la clase común deben ser diferentes, lo cual no es más que una utilización predefinida de la notación de restricción que emplea la restricción estándar de solapamiento.

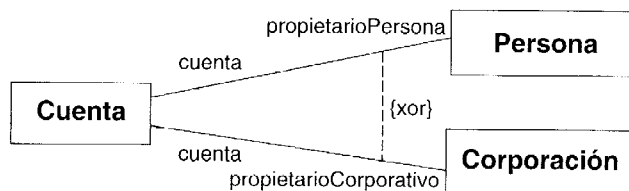


Figura 13.18 Asociación Xor

Discusión

No es necesario ponerle nombre a una asociación. Normalmente, los nombres de rol resultan más convenientes, ya que proporcionan nombres para la navegación y la generación de código a la vez que evitan el problema de saber en qué sentido leer el nombre. Si tiene nombre, debe ser único dentro de su paquete, pero si no lo tiene y hay más de una asociación entre un par (o conjunto) de clases, tienen que existir nombres de rol para distinguir las asociaciones. Si entre dos clases sólo hay una asociación, son suficientes los nombres de las clases para identificarla.

Se puede argumentar que los nombres de asociación son más útiles cuando el concepto tiene un nombre en el mundo real, tales como **Trabajo** o **Matrimonio**. Cuando un nombre de asociación está "dirigido" para ser leído en una determinada dirección, es generalmente más sencillo utilizar nombres de rol, que nunca son ambiguos se lean en la dirección que se lean.

Véase enlace transitorio, donde se discute sobre el modelado de relaciones de instancia que existen sólo durante la ejecución de un procedimiento.

Véase también composición, donde se muestra un ejemplo de generalización en que intervienen dos asociaciones.

Elementos estándar

implicit, persistence, xor.

asociación binaria

Asociación que se produce exactamente entre dos clases.

Véase también asociación, asociación n -aria.

Semántica

Una asociación binaria es una asociación con exactamente dos extremos, que es de lejos la asociación más común de todas. Como un extremo de una asociación binaria tiene otro extremo simple, las asociaciones binarias son particularmente útiles para especificar rutas de navegación entre objetos. Una asociación es navegable en un determinado sentido si puede atravesarse en ese sentido. Otras propiedades, como la multiplicidad, están definidas para asociaciones n -arias, pero son más intuitivas y útiles en las asociaciones binarias.

Notación

Una asociación binaria se representa mediante una ruta continua que conecta dos símbolos de clase. Se pueden asociar adornos a cada extremo y situar el nombre de la asociación cerca de la ruta, pero lo suficientemente lejos de los extremos como para que no haya confusiones con los nombres de rol. La notación de una asociación binaria es la misma que la de una asociación

n -aria excepto en que se suprime el rombo central. Sin embargo, las asociaciones binarias pueden tener adornos que no son aplicables a las asociaciones n -arias, como la posibilidad de navegación.

Para más detalles, véase asociación.

asociación de comunicación

Asociación que describe una relación de comunicación entre instancias de los elementos conectados. En una vista de despliegue, es una asociación entre nodos que implica una comunicación. En un modelo de casos de uso, una asociación entre un caso de uso y un actor constituye una asociación de comunicación.

Véase actor, caso de uso.

asociación n -aria

Denota una asociación entre tres o más clases. Por contrastar con: asociación binaria.

Semántica

Toda instancia de la asociación es una n -tupla de valores pertenecientes a las respectivas clases. Una clase puede aparecer en más de una posición de la asociación. Una asociación binaria es un caso especial que posee su propia notación simplificada y ciertas propiedades adicionales (tales como la navegabilidad) que carecen de sentido (o son inútilmente complicadas) para una asociación n -aria.

La multiplicidad de las asociaciones n -arias se puede especificar pero resulta menos evidente que la multiplicidad binaria. La multiplicidad del extremo de una asociación representa el número potencial de valores que puede haber en ese extremo, cuando están fijados los valores de los otros $n-1$ extremos. Obsérvese que esta definición es compatible con la multiplicidad binaria.

La agregación (incluyendo la composición) sólo tiene significado para las asociaciones binarias. Una asociación n -aria no puede contener el símbolo de agregación ni el de composición en ningún rol.

No existe diferencia semántica entre una asociación binaria y una asociación n -aria con dos extremos, independientemente de la representación. Una asociación con dos extremos se considera una asociación binaria y una que tenga más de dos extremos se considera asociación n -aria.

Notación

Una asociación n -aria se representa mediante un rombo grande (esto es, grande en comparación con el terminador de una ruta), existiendo una ruta desde el rombo hasta cada una de las clases participantes. El nombre de la asociación (si existe) se muestra junto al rombo. Pueden

aparecer adornos junto al extremo de cada ruta, tal como sucedería con una asociación binaria. Se puede indicar la multiplicidad, pero no se permiten los calificadores ni las agregaciones.

Se puede asociar un símbolo clase de asociación al rombo mediante una línea discontinua. Esto indica una asociación n -aria que posee atributos, operaciones y/o asociaciones.

Ejemplo

La Figura 13.19 muestra el registro de un equipo en las distintas temporadas de un determinado guardameta. Se supone que el portero puede ser traspasado durante la temporada y por tanto puede tener un registro con distintos equipos. En un libro de registros, cada enlace sería una línea diferente.

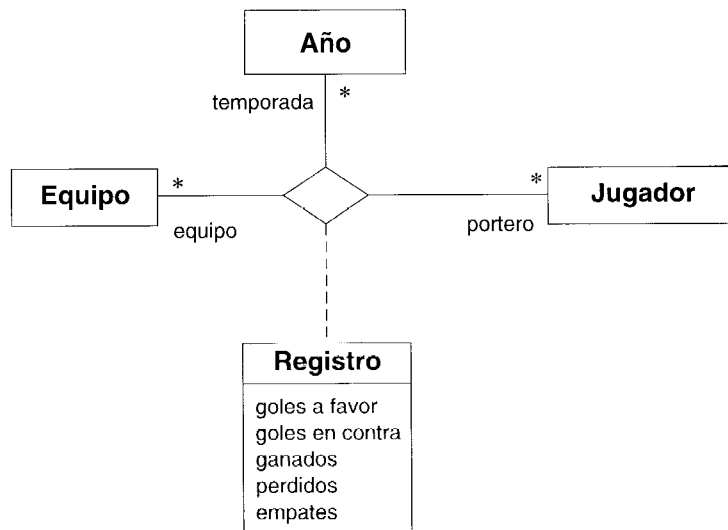


Figura 13.19 Una asociación ternaria que es además una clase asociación

Reglas de estilo

Normalmente, las líneas se dibujan saliendo de los vértices del rombo o desde el punto medio de una cara.

Discusión

En una asociación n -aria, la multiplicidad se define con respecto a los otros $n-1$ extremos. Por ejemplo, dada una asociación ternaria entre clases (A, B, C), la multiplicidad del extremo C indica el número de objetos C que pueden aparecer asociados a una pareja particular de objetos A y B. Si la multiplicidad de esta asociación es (muchos, muchos, uno) entonces para cada posible pareja (A, B) existe un único valor de C. Para una pareja dada (B, C) puede haber

muchos valores de A, sin embargo; de modo individual muchos valores de A, B y C podrían participar en la asociación. En una asociación binaria esta regla se reduce a la multiplicidad de cada extremo, que se define con respecto al otro extremo.

No tiene sentido definir la multiplicidad con respecto a un solo extremo (tal como han propuesto algunos autores) porque la multiplicidad sería muchos para cualquier asociación n -aria significativa. De no ser así, la asociación se podría fragmentar en una asociación binaria entre la clase única y una clase asociación que incluyera todas las clases restantes, mejorando así tanto la precisión como la eficiencia de la implementación. En general, lo mejor es evitar las asociaciones n -arias porque las asociaciones binarias son más fáciles de implementar y hacen posible la navegación. En general, las asociaciones n -arias sólo son útiles cuando se necesitan todos los valores para determinar un enlace de forma única. Una asociación n -aria se va a implementar en casi todas las ocasiones como una clase entre cuyos atributos se cuentan punteros de los objetos participantes. La ventaja de modelarla como una asociación es la restricción de que no puede haber enlaces duplicados dentro de una asociación.

Considere el ejemplo de un alumno que asiste a un curso de un profesor a lo largo de un curso (Figura 13.20). El alumno no va a seguir el mismo curso de más de un profesor, pero un alumno puede asistir a más de un curso de un único profesor, y el profesor puede impartir más de un curso. Las multiplicidades se muestran en el diagrama. La multiplicidad del **Profesor** es opcional (0..1); las otras multiplicidades son muchas (0..*).

Todo valor de multiplicidad es relativo a una pareja de objetos de otros extremos. Para toda pareja (curso, alumno) existe cero o un profesor. Para toda pareja (alumno, profesor) existen muchos cursos. Para toda pareja (curso, profesor) existen muchos alumnos.

Obsérvese que si esta asociación se materializa en una clase entonces sería posible tener más de una copia de la misma combinación (alumno, curso, profesor) lo cual no resulta deseable.

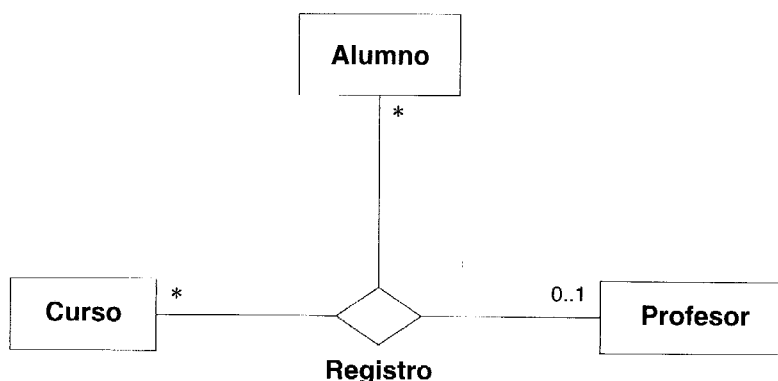


Figura 13.20 Multiplicidad en una asociación n -aria

atómico/a

Acción u operación cuya ejecución debe completarse como un todo, por lo que no puede ejecutarse parcialmente ni ser finalizada por un evento externo. Normalmente, las operaciones

atómicas son pequeñas y sencillas, como asignaciones, cálculos aritméticos sencillos u operaciones de tipo cadena. Un cómputo atómico ocurre en un punto definido de la secuencia de ejecución.

Véase también acción, actividad, ejecución a completar.

Semántica

Globalmente, el sistema puede realizar muchas acciones simultáneamente. Cuando decimos que una acción es atómica, eso no implica que el sistema completo sea atómico. El sistema puede procesar interrupciones del hardware y compartir el tiempo entre diversas acciones, ya que una acción es atómica sólo dentro de su hilo de control. Una vez iniciada, debe completar su ejecución sin interactuar con otras acciones que estén activas al mismo tiempo. El sistema puede procesar interrupciones y eventos, pero eso no debe afectar a la operación atómica, ya que dichas acciones no deberían usarse como un mecanismo para transacciones largas. Su duración debería ser breve comparada con el tiempo de respuesta necesario para responder a eventos externos porque, si no, el sistema podría ser incapaz de responder a tiempo.

atributo

Un atributo es la descripción de una ranura con nombre de un tipo especificado en una clase; cada objeto de la clase tiene un valor independiente para el atributo.

Semántica

Un atributo es una ranura con nombre dentro de un clasificador que describe los valores que pueden tener las instancias del clasificador. Todas las instancias del clasificador o de uno de sus descendientes tienen una ranura que guarda un valor del tipo dado. Todas las ranuras son distintas e independientes de las demás (excepto en el caso de los atributos con alcance de clase, que se describen más adelante). Cuando se lleva a cabo una ejecución, el valor de una ranura dentro de una instancia puede remplazarse por otro del mismo tipo, ya que los atributos son modificables.

Un clasificador forma un espacio de nombres para sus atributos, aunque también forman parte del espacio de nombres los pseudoatributos, los nombres de rol de las asociaciones que salen del clasificador y los discriminadores de generalizaciones en las que participe el clasificador.

Estructura

Un atributo tiene los siguientes componentes principales, que se describen con más detalle en sus propias entradas:

alcance de propietario En una clase, el valor descrito por un atributo puede ser distinto en cada objeto o compartido por todos los objetos. En el primer caso,

se trata de un atributo con alcance de instancia, mientras que en el último se trata de un atributo con alcance de clase. La mayoría de los atributos tienen alcance de instancia, esto es, mantienen información de estado acerca de un objeto en concreto. Los atributos con alcance de clase guardan información sobre una clase entera, ya que hay un único valor de la ranura para toda la clase.

Mientras que un atributo con alcance de instancia es una descripción de un valor sin existencia hasta que no se instancia el objeto, un atributo con alcance de clase representa la declaración de un valor discreto individual que existe a lo largo de toda la vida de un sistema.

alcance destino	El valor de un atributo puede ser una instancia o una clase. En el primer caso sería un alcance de instancia (valor por defecto), mientras que en el segundo es un alcance de clase, caso raro que normalmente implica alguna forma de metamodelado.
multiplicidad	El posible número de valores del atributo que pueden existir simultáneamente. El valor más común, “exactamente uno” denota un atributo escalar. El valor “cero o uno” denota un atributo con un valor opcional. La no existencia de valor es distinta de cualquier valor del dominio del tipo del atributo (en otras palabras, la ausencia de valor es distinta del valor cero, pues significa conjunto vacío). Otras multiplicidades denotan atributos que pueden tener múltiples valores. Si la multiplicidad no es un entero simple, es porque el número de valores del atributo puede variar. La multiplicidad “muchos” denota un conjunto ilimitado de valores.
nombre	El nombre de un atributo, una cadena que debe ser única dentro de la clase y sus antecesores. También debe ser única entre los nombres de los roles de asociación alcanzables desde la clase.
tipo	Designa una clase o tipo de dato del que son instancias los valores de la ranura. Un valor puede ser una instancia de un descendiente de la clase o tipo de dato dado.
valor inicial	Expresión que indica el valor de un atributo en un objeto justo después de haber sido inicializado. La expresión es una cadena de texto, junto con el nombre del lenguaje utilizado para evaluar la expresión. La expresión se evalúa en el contexto del lenguaje cuando se instancia el objeto. Véase expresión para más detalles. El valor inicial es opcional; si no lo hay, el modelo estático no especifica el valor para los objetos nuevos, aunque otra parte del modelo global puede proporcionar dicha información.

Obsérvese que un procedimiento de inicialización explícito, como un constructor, puede reemplazar una expresión de valor inicial.

El valor inicial de un atributo con alcance de clase se utiliza para inicializarlo una vez al principio de la ejecución. UML no especifi-

	ca el orden relativo de inicialización de los diferentes atributos con alcance de clase.						
variabilidad	Indica si el valor de la ranura puede cambiar tras la inicialización. Se utiliza una enumeración cuyo valor por defecto es changeable . Los posibles valores son: <table> <tr> <td>changeable</td><td>(modificable): no hay restricciones para modificarlo. Valor por defecto.</td></tr> <tr> <td>addOnly</td><td>(sólo añadir): para atributos con multiplicidad mayor que uno, se pueden añadir valores adicionales al conjunto de valores del atributo, pero una vez creado, un valor no puede ser modificado ni eliminado.</td></tr> <tr> <td>frozen</td><td>(congelado): el valor no se puede modificar una vez inicializado el objeto. No se pueden añadir valores adicionales a un conjunto de valores.</td></tr> </table>	changeable	(modificable): no hay restricciones para modificarlo. Valor por defecto.	addOnly	(sólo añadir): para atributos con multiplicidad mayor que uno, se pueden añadir valores adicionales al conjunto de valores del atributo, pero una vez creado, un valor no puede ser modificado ni eliminado.	frozen	(congelado): el valor no se puede modificar una vez inicializado el objeto. No se pueden añadir valores adicionales a un conjunto de valores.
changeable	(modificable): no hay restricciones para modificarlo. Valor por defecto.						
addOnly	(sólo añadir): para atributos con multiplicidad mayor que uno, se pueden añadir valores adicionales al conjunto de valores del atributo, pero una vez creado, un valor no puede ser modificado ni eliminado.						
frozen	(congelado): el valor no se puede modificar una vez inicializado el objeto. No se pueden añadir valores adicionales a un conjunto de valores.						
visibilidad	Indica si el atributo es visible desde otras clases. Se representa mediante una enumeración donde pueden aparecer las palabras clave public , private o protected . Se pueden añadir valores adicionales para modelar ciertos lenguajes de programación.						

Notación

Un atributo se representa mediante una cadena de texto que puede dividirse en varias propiedades. La sintaxis por defecto es:

$$\overline{\text{«estereotipo»}}_{opc} \text{ visibilidad}_{opc} \text{ nombre multiplicidad}_{opc} : \text{tipo}_{opc}$$

$$= \text{valor-inicial}_{opc} \{ \text{cadena-de-propiedades} \}_{opc}$$

Visibilidad. La visibilidad se representa mediante una marca de puntuación, aunque también se puede representar utilizando una palabra clave dentro de la cadena de propiedades, que se utiliza sobre todo para incluir opciones definidas por el usuario o dependientes del lenguaje. Las opciones predefinidas son:

+ (public)	Cualquier clase que vea la clase ve también sus atributos.
# (protected)	Sólo la propia clase o sus descendientes pueden ver los atributos.
– (private)	Sólo la propia clase puede ver los atributos.

Nombre. El nombre se representa mediante un identificador de tipo cadena.

Tipo. El tipo se representa mediante una expresión de cadena que denota un clasificador. El nombre de una clase o tipo de datos es una expresión legítima de tipo cadena que indica que los valores de un atributo deben ser de un tipo dado. La sintaxis adicional del tipo depende del

lenguaje de la expresión. Cada lenguaje tiene una sintaxis de construcción de nuevos tipos de datos a partir de los datos existentes. Por ejemplo, C++ tiene sintaxis para los punteros, para los arrays y para las funciones. Ada también tiene sintaxis para los subrangos. El lenguaje de la expresión es parte del modelo interno, pero normalmente no se indica en un diagrama, ya que se supone conocida en todo el diagrama u obvio a partir de su sintaxis.

La cadena de tipo se puede suprimir (pero sigue presente en el modelo).

Multiplicidad. La multiplicidad se representa mediante una expresión de multiplicidad encerrada entre corchetes y situada después del nombre del atributo. Si la multiplicidad es “exactamente uno”, se puede omitir la expresión, incluyendo los corchetes. Esto indica que cada objeto tiene exactamente una ranura para guardar un valor del tipo especificado. Si no es así, debe mostrarse la multiplicidad. Véase multiplicidad, donde hay una completa discusión sobre la sintaxis. Por ejemplo:

colores [3]:Saturación	Un array de 3 saturaciones
puntos [2..*]:Punto	Un array de dos o más puntos

Obsérvese que una multiplicidad de 0..1 abre la posibilidad de valores nulos (la ausencia de valor como oposición a un valor concreto dentro del rango). Un valor nulo no es un valor del dominio de la mayoría de los tipos de datos: extiende el dominio con un valor extra que se encuentra fuera del mismo. Sin embargo, en el caso de los punteros el valor nulo es a menudo parte de la implementación (aunque, incluso entonces, se usa normalmente por convenio: por ejemplo el valor 0 en C o C++ es un convenio de direccionamiento de memoria). La siguiente declaración permite distinguir entre el valor *null* y la cadena vacía, una distinción presente en C++ y en otros lenguajes:

nombre [0..1]:String	Si no tiene nombre, es un valor nulo.
----------------------	---------------------------------------

Valor inicial. El valor inicial se representa mediante una cadena. El lenguaje de evaluación normalmente no se indica explícitamente, aunque está presente en el modelo. Si no hay valor inicial, se omiten tanto la cadena como el signo igual. Si la multiplicidad del atributo incluye el valor 0 (es decir, opcional) y no se le ha dado explícitamente ningún valor, el atributo comienza teniendo un valor nulo (cero repeticiones).

Variabilidad. El valor de modificación se representa mediante una palabra clave que representa el valor elegido. Si no se elige ningún valor, se toma por defecto **changeable**.

Valor etiquetado. Un atributo puede tener asociados cero o más valores etiquetados (como cualquier elemento del modelo). Cada valor etiquetado se representa de la forma etiqueta = valor, donde etiqueta es el nombre de una etiqueta y valor es un valor literal. Los valores etiquetados se incluyen con las palabras clave de las propiedades como una lista entre corchetes de propiedades separadas por comas.

Alcance. El alcance de clase de un atributo se representa subrayando la cadena que expresa el tipo y nombre. Si no está subrayado, el atributo tiene alcance de instancia. La justificación de la notación es que un atributo con alcance de clase es un valor en el sistema en ejecución, al igual que un objeto es un valor de instancia, por lo que ambos deben subrayarse.

atributo-con-alcance-de-clase

La Figura 13.21 muestra la declaración de algunos atributos.

+tamaño: Área = (100,100)	público, valor inicial
#visibilidad: Boolean = invisible	protegido, valor inicial
+tamaño-por-defecto: Rectángulo	público
tamaño-máximo: Rectángulo	alcance de clase
-xptr: XWindowPtr {requisito = 4.3}	privado, valor etiquetado

Figura 13.21 Atributos

Opciones de presentación

Sintaxis del lenguaje de programación. La sintaxis de la cadena de un atributo puede ser la de un lenguaje de programación, como C++ o Smalltalk. Se pueden incluir en la cadena propiedades etiquetadas específicas.

Reglas de estilo

Los nombres de atributo se representan con un tipo de letra normal.

Discusión

Se utiliza una sintaxis similar para representar calificadores, parámetros de plantillas, parámetros de operaciones, y otros, aunque algunos de ellos omiten ciertos términos.

Obsérvese que un atributo es equivalente semánticamente a una asociación de composición, aunque el uso y la intención son totalmente diferentes. Utilice los atributos para tipos de datos, esto es, para valores sin identidad. Utilice las asociaciones para clases, o lo que es lo mismo, para valores con identidad. La razón es que en los objetos con identidad es importante ver la relación en ambas direcciones, mientras que en los tipos de datos, el tipo normalmente está subordinado al objeto y no sabe que existe.

Elementos estándar

persistence.

autotransición

Transición en la cual el estado origen y el estado destino son el mismo. Se considera un cambio de estado. Cuando se dispara, se sale del estado origen y se vuelve a entrar en él, de tal modo que se invocan las acciones de entrada y las de salida. No es equivalente a una transición interna, en la cual no se produce un cambio de estado.

become (se convierte en)

Tipo de dependencia de flujo, utilizada en una interacción, en la cual el objeto destino representa una nueva versión del objeto origen y a partir de entonces lo reemplaza.

Véase también clase en un estado, copia, localización.

Semántica

Una dependencia *become* es un tipo de dependencia de flujo que representa la derivación de un objeto a partir de otro dentro de una interacción. Después de ejecutarse un flujo *become*, el nuevo objeto reemplaza al original dentro del cómputo. Normalmente no es necesario utilizar esta relación para representar únicamente un cambio en el valor de un objeto. Por otra parte, esta relación es útil para representar un cambio cualitativo en un objeto, como un cambio de estado, un cambio de clase o un cambio de localización. En estas situaciones, el modelo contiene dos versiones del objeto, pero la relación *become* muestra que son realmente el mismo objeto, esto es, que tienen la misma identidad.

Una transición *become* dentro de una interacción puede tener un número de secuencia para indicar cuándo sucede con respecto a otras acciones.

Notación

Un flujo *become* se representa mediante una flecha discontinua con el estereotipo «**become**» que sale de la primera versión del objeto y apunta a la última. En una interacción, la flecha puede tener un número de secuencia cuando el cambio se produce con relación a otras acciones. Las transiciones *become* pueden aparecer en diagramas de colaboración, en diagramas de secuencia y en diagramas de actividades.

En un diagrama de actividades, la transición *become* se puede representar mediante una flecha discontinua desde o hacia el símbolo de flujo de objetos. Se puede omitir la palabra clave *become*.

Ejemplo

La Figura 13.22 muestra una orden para abrir un icono de directorio cerrado en un escritorio, seguido por otra que ordena los elementos del recién abierto directorio. El directorio se muestra dos veces como objeto de una clase en un estado, con una transición *become* entre las dos versiones.

La Figura 13.142 muestra un diagrama de despliegue en el que un objeto migra entre dos nodos.

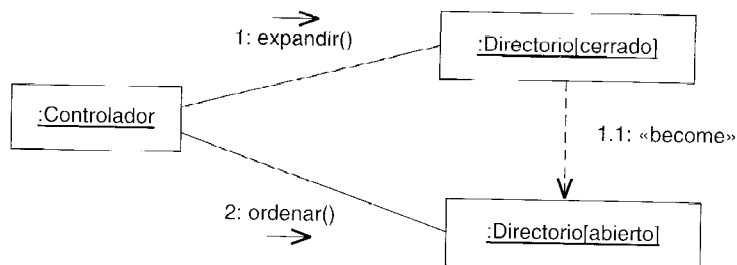


Figura 13.22 Flujo become

bien formado

Denota un modelo que está construido correctamente, que satisface todas las reglas predefinidas y establecidas por el modelo. Este modelo tiene una semántica significativa. Un modelo que no sea bien formado recibe el nombre de mal formado.

bifurcación

Elemento de una máquina de estados en el que un único disparador tiene más de un posible resultado, cada uno con su propia condición de guarda.

Véase también división, estado de unión o conjunción, unión.

Semántica

Si un mismo evento puede tener diferentes efectos dependiendo de las distintas condiciones de guarda, puede modelarse entonces como transiciones separadas con el mismo evento disparador. Sin embargo, en la práctica es conveniente permitir que un disparador simple dirija múltiples transiciones. Esto es especialmente cierto en el caso tan común en que las condiciones de guarda cubren todas las posibilidades, caso en el que se garantiza que una ocurrencia del evento disparará una de las transiciones. Una *bifurcación* es una parte de la transición que divide la ruta de la misma en dos o más segmentos, cada uno de ellos con una condición de guarda. El evento disparador se sitúa en el segmento común de la transición (el primer segmento). La salida de un segmento de una bifurcación puede conectarse con la entrada de otra bifurcación formando un árbol, de forma que cada ruta a través del árbol represente una transición diferente. La conjunción de todas las condiciones de una ruta de una transición es equivalente a una condición simple que se evalúa antes de que se dispare la transición. Una transición se dispara en un único paso, a pesar de su apariencia de conjunto de bifurcaciones, ya que el árbol no es más que un convenio de notación.

En un grafo de actividades, las actividades que salen de un estado de actividad son, generalmente, transiciones de finalización, esto es, carecen de eventos disparadores explícitos, y se disparan implícitamente cuando finaliza la actividad del estado. Si existen condiciones de guarda o bifurcaciones, es importante que cubran todas las posibilidades para que se dispare alguna transición, pues si no se congelaría la ejecución del grafo de actividades, ya que las transiciones de salida nunca volverían a estar disponibles.

Notación

Una bifurcación se puede representar repitiendo un evento disparador en los arcos de muchas transiciones con diferentes condiciones de guarda. También con transiciones de finalización, como en los diagramas de actividades.

Sin embargo, es más conveniente representar las bifurcaciones utilizando un rombo al que se conecta la punta de flecha de una transición. La flecha de la transición se etiqueta con el evento disparador, si lo hay, pero no debería llevar acciones asociadas. Las acciones irían en el segmento final de la transición.

De un rombo pueden salir dos o más flechas, cada una de las cuales se etiqueta con una condición de guarda. Se puede utilizar la palabra reservada **else** como condición de guarda, que será verdadera cuando las demás condiciones explícitas de guarda sean falsas. El final de una

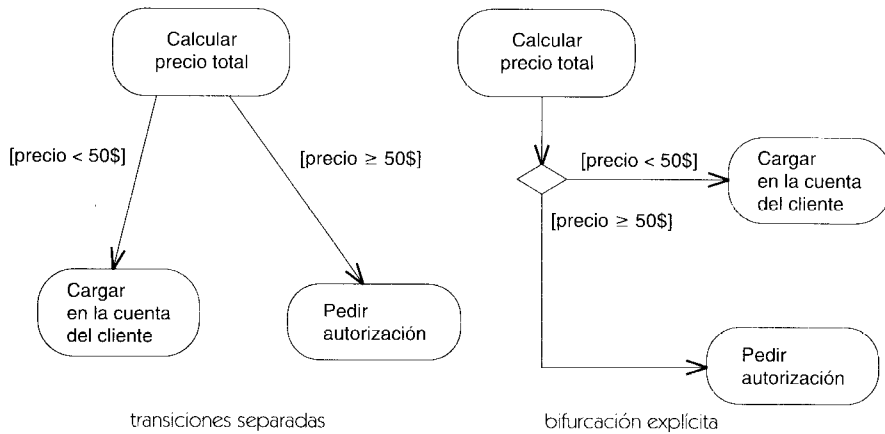


Figura 13.23 Dos formas de representar una bifurcación

flecha se puede conectar con otra bifurcación o con otro estado, y cuando se conecte a un estado, podrá llevar una acción asociada.

El efecto de árbol de bifurcaciones es el mismo que el que se produce expandiendo el árbol en un arco de transición separado para cada ruta del árbol, donde todos comparten el mismo evento disparador, pero cada uno de los cuales tiene su propia conjunción de condiciones de guarda, acciones y estados finales. La Figura 13.23 muestra dos formas de representar la misma situación.

Obsérvese que se puede utilizar también el símbolo del rombo para la reunificación (operación inversa de la bifurcación), donde dos rutas separadas se unen, tal y como se muestra en la Figura 13.24. En el caso de la reunificación habrá dos o más flechas de entrada y una sola de salida. No son necesarias las condiciones de guarda.

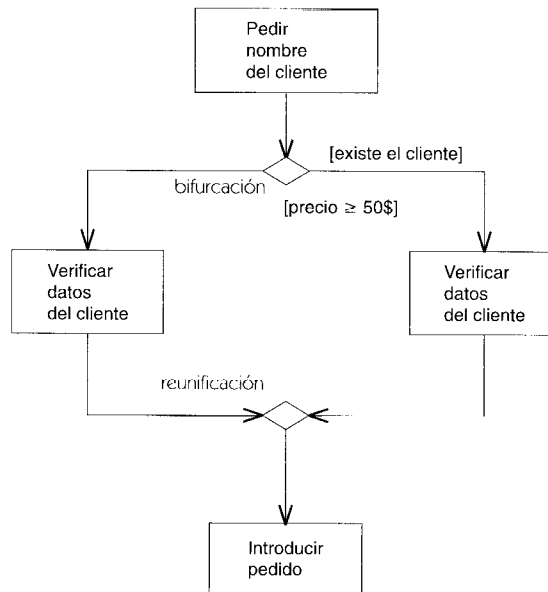


Figura 13.24 Bifurcación y reunificación

bind (ligar)

Palabra clave para expresar una dependencia de ligadura en la notación.

Véase ligadura.

booleano/a

Enumeración cuyos valores son **true** y **false**.

cadena

Secuencia de caracteres de texto. Los detalles de la representación de cadenas dependen de la implementación y pueden incluir conjuntos de caracteres que admitan gráficos y caracteres internacionales.

Semántica

Hay muchas propiedades semánticas, especialmente los nombres, que tienen una cadena como valor. Una cadena es una secuencia de caracteres pertenecientes a algún conjunto de caracteres adecuado, que se emplea para visualizar información relativa al modelo. Entre los juegos de caracteres se pueden incluir alfabetos y caracteres no latinos. UML no especifica la codificación de cadenas pero supone que la codificación es suficientemente general para permitir cualquier utilización razonable. En principio, la longitud de las cadenas debería ser ilimitada; toda limitación práctica debería ser suficientemente grande para que no resulte restrictiva. Las cadenas deberían incluir la posibilidad de admitir mostrar caracteres de diferentes idiomas humanos. Los identificadores (los nombres) deberían estar formados únicamente por caracteres de un conjunto finito de caracteres. Los comentarios y clases similares de cadenas descriptivas sin contenido semántico puede contener otras clases de medios de elementos, tales como diagramas, grafos, imágenes o segmentos de vídeo y otros tipos de documentos incrustados.

Notación

Una cadena gráfica es un elemento primitivo de notación con cierta flexibilidad de implementación. Se supone que se trata de una secuencia lineal de caracteres escrita en algún idioma, con la posible inclusión de documentos incrustados de distintas clases. Resulta deseable admitir la utilización de diferentes idiomas humanos, pero se dejan los detalles a las herramientas de edición para que sean éstas las que los implementen. Las cadenas gráficas pueden ir en líneas individuales, en listas o bien pueden ser etiquetas asociadas a otros símbolos.

Las cadenas se utilizan para mostrar propiedades semánticas que tienen cadenas como valores y también se emplean para codificar los valores de otras propiedades semánticas para su visualización. La correspondencia entre cadenas semánticas y cadenas de notación es directa.

La correspondencia de otras propiedades con las cadenas de notación está controlada por las gramáticas, que se describen en los artículos correspondientes a los distintos elementos. Por ejemplo, la notación de visualización para los atributos codifica el nombre, valor, visibilidad y alcance en una única cadena de visualización.

Es posible realizar extensiones no canónicas de las codificaciones —por ejemplo, se podría visualizar un atributo empleando la notación de C++—. Algunas de estas codificaciones pueden perder información del modelo, sin embargo, las herramientas deberían admitirlas como opciones seleccionables por el usuario, manteniendo ciertamente la notación canónica de UML.

El estilo y tamaño del tipo de letra son marcadores gráficos, que normalmente serán independientes de la cadena en sí. Se pueden codificar para distintas propiedades del modelo, algunas de las cuales se sugieren en este documento; otras se dejan al albedrío de la herramienta o del usuario. Por ejemplo, la cursiva muestra clases y operaciones abstractas y el subrayado muestra características con alcance de clase.

Las herramientas pueden tratar de distintas maneras las cadenas más largas; se pueden truncar hasta un tamaño dado, se puede hacer un salto de línea automático y se pueden insertar barras de desplazamiento. Se supone que existe alguna manera de obtener la cadena completa si se desea.

calificador

Denota una ranura de un atributo o lista de atributos en una asociación binaria, en la cual los valores de los atributos seleccionan un único objeto relacionado o un conjunto de objetos relacionados dentro de todo el conjunto de objetos relacionados con un objeto por esa asociación. Se trata de un índice para recorrer una asociación.

Véase clase asociación, extremo de asociación.

Semántica

Toda asociación binaria hace corresponder a un objeto un conjunto de objetos relacionados. En algunas ocasiones resulta deseable seleccionar un objeto del conjunto, proporcionando un valor que distingue a los objetos del conjunto. Este valor podría ser un atributo de la clase destino. En general, sin embargo, el valor selector podría ser parte de la asociación en sí, un atributo de la asociación cuyo valor es proporcionado por el creador cuando se añade un nuevo enlace a la clase asociación. Un atributo como éste, en una asociación binaria, recibe el nombre de calificador. Un objeto, junto con un valor de calificador, determina un objeto único relacionado con él o bien (con menos frecuencia) un subconjunto de objetos relacionados. El valor califica la asociación. En un contexto de implementación, a estos atributos se les da el nombre de valores de índice.

Los calificadores se utilizan para seleccionar un objeto u objetos del conjunto de objetos relacionados con un objeto (que se denomina *objeto calificado*) mediante una asociación (Figura 13.25). El objeto seleccionado por el valor del calificador se denomina *objeto destino*.

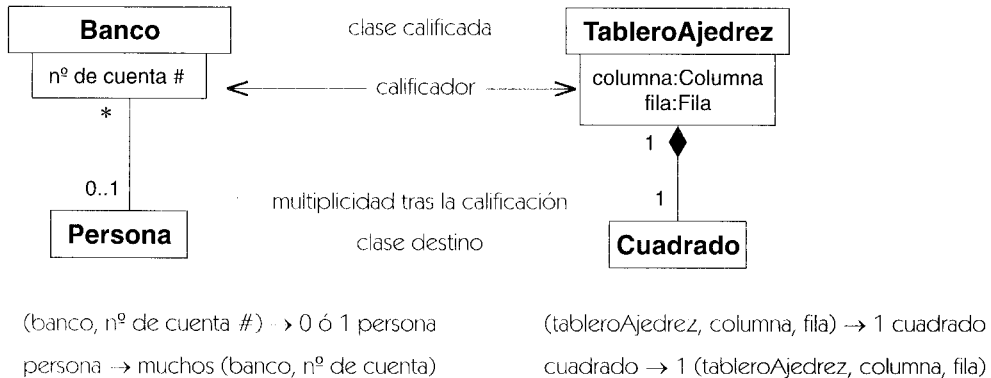


Figura 13.25 Asociaciones calificadas

Un calificador siempre actúa sobre una asociación cuya multiplicidad es *muchos* en la dirección del destino. En el caso más sencillo, cada valor del calificador selecciona un único objeto del conjunto destino de objetos relacionados. En otras palabras, un objeto calificado y un valor de calificador producen un único objeto destino relacionado. Dado un objeto calificado, cada valor del calificador se corresponde con un único objeto destino.

Hay muchas clases de nombres que son calificadores. Estos nombres dentro de un contexto se corresponden con un único valor. El objeto calificado proporciona el contexto y el objeto destino es el resultado. Toda ID u otro código único es un calificador; su propósito es seleccionar un valor de modo único. Las matrices se pueden modelar como asociaciones calificadas. La matriz es el objeto calificado, el índice de la matriz es el calificador y el elemento de la matriz es el objeto destino. Para una matriz, el tipo calificador es un intervalo de enteros.

Se puede emplear un calificador en una expresión de navegación para seleccionar un subconjunto de objetos relacionados con un objeto a través de una asociación, a saber, aquellos que tengan un valor concreto para el valor del atributo calificador o lista de valores. El calificador hace las veces de selector dentro del conjunto de objetos relacionados por la asociación. Descompone el conjunto en subconjuntos por valores del calificador. En la mayoría de los casos, el propósito del calificador es seleccionar un objeto único dentro del conjunto de objetos relacionados de tal modo que una asociación calificada se comporta como una tabla de búsqueda.

Estructura

Calificador. Un atributo calificador es una parte opcional del extremo de una asociación binaria. El calificador califica a la clase asociada al extremo de la asociación. Un objeto de la clase y un valor del calificador seleccionan un objeto o conjunto de objetos de la clase situada al otro extremo de la asociación binaria. Es posible que los dos extremos de la asociación binaria tenga calificadores, pero no suele suceder.

Un calificador es un atributo o lista de atributos de una asociación. Cada atributo posee nombre y tipo pero no valor inicial, porque los calificadores no son objetos independientes; el valor de cada calificador debe ser explícito siempre que se añade un enlace a la asociación.

Los calificadores no se utilizan en asociaciones *n*-arias.

Multiplicidad. La multiplicidad de la relación calificada se sitúa en el extremo opuesto de la asociación binaria respecto al calificador. (La regla mnemotécnica es que la clase calificada y el calificador, juntos, forman un valor compuesto que está relacionado con la clase destino.) En otras palabras, el calificador se asocia al extremo “próximo” de la asociación y la multiplicidad y el nombre de rol se asocian al extremo “lejano”.

La multiplicidad asociada al extremo destino de la asociación denota el número de objetos destino que podrían ser seleccionados por una pareja (objeto origen, valor de calificador). Entre los valores comunes de multiplicidad se cuentan **0..1** (se puede seleccionar un único valor pero no todo posible valor del calificador selecciona un valor necesariamente), **1** (todo posible valor del calificador selecciona un único objeto destino, luego el dominio de valores del calificador debe ser finito) y ***** (el valor del calificador es un índice que descompone el objeto destino en subconjuntos).

En la mayoría de los casos, la multiplicidad es cero-o-uno. Esta opción significa que un objeto y un calificador pueden producir, como máximo, un único objeto relacionado. Una multiplicidad de uno significa que todo posible valor del calificador produce exactamente un objeto. Evidentemente, esto requiere que el tipo del calificador sea un dominio finito (al menos en la implementación para un computador). Esta multiplicidad puede ser útil para hacer corresponder tipos finitos enumerados; por ejemplo, un **Píxel** calificado por un **ColorPrimario** (enumeración de rojo, verde y azul) produciría el triplete de valores rojo-verde-azul para todos los píxeles de una imagen.

La multiplicidad de una asociación no calificada no se enuncia explícitamente. Pero suele suponerse que es muchos, o por lo menos mayor que uno. En caso contrario, no habría necesidad de tener un calificador.

Una multiplicidad de muchos en una asociación calificada no tiene un impacto semántico significativo, porque el calificador no reduce la multiplicidad del conjunto destino. Esta multiplicidad representa una sentencia de diseño consistente en que debe proporcionarse un índice para recorrer la asociación. En tal caso, el calificador descompone el conjunto de objetos destino en subconjuntos. Semánticamente, esto no añade nada más allá de tener un atributo de asociación, que también descompone (implícitamente) los enlaces. La connotación de diseño de un calificador en un modelo de diseño es que el recorrido debería ser eficiente, esto es, que no debe requerir una búsqueda lineal entre todos los valores destino. Normalmente se implementa mediante algún tipo de tabla de búsqueda. Un índice de una base de datos o de una estructura de datos se modela correctamente como un calificador.

En la dirección inversa a través de una asociación calificada (esto es, yendo desde la clase destino hacia el objeto calificado), la multiplicidad indica el número de parejas (objeto calificado, calificador) que se pueden relacionar con un objeto destino, no el número de objetos calificados. En otras palabras, si varias parejas (objeto calificado, calificador) se corresponden con el mismo objeto destino, entonces la multiplicidad inversa será muchos. Una multiplicidad inversa de uno entre destino y calificador significa que existe exactamente un emparejamiento de objeto calificado y valor calificador que está relacionado con el objeto destino.

Notación

Los calificadores se representan mediante un pequeño rectángulo asociado al extremo de una ruta de asociación entre el segmento final de la ruta y el símbolo de la clase calificada. El

rectángulo calificador forma parte de la ruta de asociación, no de la clase. El calificador está asociado a la clase que califica, esto es, un objeto de la clase calificada junto con un valor del calificador selecciona de modo único un conjunto de objetos de la clase destino situada en el otro extremo de la asociación.

Los atributos de calificador se enumeran dentro del cuadro calificador. Puede haber uno o más atributos en la lista. Los atributos de calificador tienen la misma notación de los atributos de clase, salvo que las expresiones de valor inicial no tienen sentido.

Opciones de presentación

Los calificadores no se pueden suprimir (proporcionan detalles esenciales cuya defecto modificaría el carácter inherente de la relación).

Las herramientas pueden utilizar una línea más fina para los rectángulos de calificador que para los rectángulos de clase, con objeto de distinguirlos claramente.

Preferiblemente, el rectángulo calificador debería ser más pequeño que el rectángulo de clase al que esté asociado, aunque esto no siempre es posible en la práctica.

Discusión

Las multiplicidades de una asociación calificada se tratan como si el objeto calificado y el calificador fueran una sola entidad, una clave compuesta. En la dirección hacia delante, la multiplicidad del extremo destino representa el número de objetos relacionados con el valor compuesto (objeto calificado + valor de calificador). En la dirección inversa, la multiplicidad describe el número de valores compuestos (objeto calificado + calificador) que están relacionados con cada objeto destino, no el número de objetos calificados relacionados con cada objeto destino. Ésta es la razón por la cual se pone el calificador en el mismo extremo de la ruta de asociación, junto al símbolo de la clase. Se puede pensar que la ruta de la asociación conecta el valor compuesto con la clase destino.

No se ha previsto especificar la multiplicidad de las relaciones no calificadas. Sin embargo, en la práctica suelen ser muchos en la dirección hacia delante. No tiene sentido tener una asociación calificada salvo que haya muchos objetos relacionados con un único objeto calificado. Para el modelado lógico, el propósito del calificador es reducir la multiplicidad a uno mediante la adición de un calificador, de tal modo que se asegure que una consulta pueda proporcionar un único valor en lugar de un conjunto de valores. La unicidad del valor del calificador suele ser frecuentemente una condición semántica crucial, que es difícil de capturar sin calificadores. Casi todas las asociaciones poseen muchas asociaciones calificadas. Hay muchos nombres que son realmente calificadores. Si un nombre es único en un determinado contexto, se trata de un calificador y el contexto debería ser identificado y modelado adecuadamente. No todos los nombres son calificadores. Los nombres de personas, por ejemplo, no son únicos. Dado que los nombres personales son ambiguos, la mayor parte de las aplicaciones de procesamiento de datos hace uso de algún tipo de número de identificación, tal como un número de cliente, un número de Seguridad Social o un número de empleado. Si una aplicación requiere la búsqueda de información o la recuperación de datos basada en claves de búsqueda, en general el modelo debería utilizar asociaciones calificadas. Todo contexto en el cual se

definan nombres o códigos de identificación para seleccionar cosas dentro de un conjunto deberá modelarse, normalmente, como una asociación calificada.

Observe que el valor del calificador es una propiedad del enlace, no del objeto destino. Considere un sistema de archivos Unix, en el cual cada directorio es una lista de entradas cuyos nombres son únicos dentro del directorio, aun cuando se pueden utilizar los mismos nombres dentro de otros directorios. Cada entrada denota un archivo, que puede ser un archivo de datos u otro directorio. Es posible tener más de una entrada que apunte a un mismo archivo. Si sucede esto, el archivo tiene varios alias. El sistema de directorios de Unix está modelado como una asociación muchos-a-uno en la cual el directorio calificado por un nombre de archivo produce un archivo. Observe que el nombre de archivo no forma parte del archivo; forma parte de la relación existente entre un directorio y un archivo. Un archivo no posee un único nombre. Puede tener múltiples nombres en otros tantos directorios (o incluso varios nombres en un mismo directorio). El nombre de archivo no es un atributo del archivo.

Una de las motivaciones más importantes para la existencia de asociaciones calificadas es la necesidad de modelar una importante situación semántica que posee una estructura de datos de implementación natural e importante. En la dirección hacia delante, una asociación calificada es una tabla de búsqueda; para un objeto calificado, cada valor del calificador produce un único objeto destino (o un valor nulo si el valor del calificador está ausente en el conjunto de valores). Las tablas de búsqueda se pueden implementar mediante estructuras de datos tales como tablas hash, árboles-b y listas ordenadas que proporcionan una eficiencia mucho mayor que las listas desordenadas. En casi todos los casos, son malos los diseños que utilizan una lista enlazada o alguna otra estructura desordenada para buscar nombres o códigos, aunque lamentablemente hay muchos programadores que las usan. El modelado de situaciones adecuadas empleando asociaciones calificadas y estructuras de datos eficientes para implementarlas resulta crucial para una buena programación.

Para un modelo lógico, tiene poco sentido tener una asociación calificada con una multiplicidad de muchos en la dirección hacia adelante, porque el calificador no añade ninguna información semántica que no pudiera mostrar un atributo de asociación. Sin embargo, en un modelo destinado al diseño de algoritmos y estructuras de datos, un calificador aporta una connotación adicional, a saber, la intención de que la selección sea eficiente. En otras palabras, una asociación calificada denota una estructura de datos indizada y optimizada para la búsqueda por valores del calificador. En este caso, puede ser útil una multiplicidad de muchos para representar un conjunto de valores al que deba poderse acceder en bloque a través de un valor de índice común, sin tener que buscar otros valores.

En general, no debe incluirse un atributo calificado como atributo de la clase destino, porque es suficiente su presencia en la asociación. En el caso de un valor índice, sin embargo, puede ser necesario tomar un valor que sea inherentemente un atributo de la clase destino y hacer que sea un valor de calificador redundante. Los valores de índice son inherentemente redundantes.

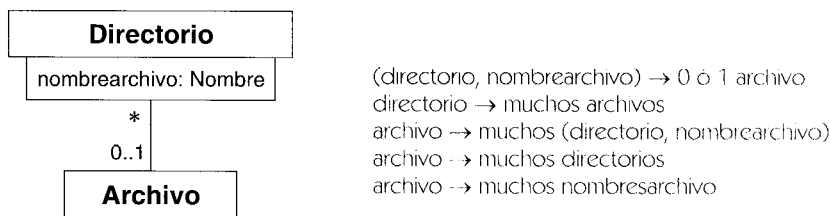


Figura 13.26 Calificador sencillo

Restricciones

Algunas situaciones complicadas no son fáciles de modelar con cualquier conjunto de relaciones no redundantes. Lo mejor es modelarlas empleando asociaciones calificadas para capturar las rutas básicas de acceso y enunciar explícitamente las restricciones adicionales. Dado que estas situaciones son poco frecuentes, estimamos que intentar incluirlas en una notación que capture directamente todas las posibles restricciones de multiplicidad no merecía la complejidad adicional.

Por ejemplo, considere un directorio en el que cada nombre de archivo identifica a un archivo único. Un archivo puede corresponder a múltiples parejas directorio-nombre de archivo. Éste es el modelo básico que se ha visto antes. El modelo se muestra en la Figura 13.26.

Ahora, sin embargo, deseamos añadir restricciones adicionales. Suponga que todo archivo debe estar en un único directorio, pero dentro de ese directorio puede tener múltiples nombres, esto es, que hay más de una forma de dar nombre al mismo archivo. Esto se podría modelar con una asociación redundante entre **Archivo** y **Directorio**, con una multiplicidad de un **Directorio** (Figura 13.27). La redundancia de las dos asociaciones se indica mediante la restricción **{igual}**, que implica que los dos elementos son el mismo pero con distintos niveles de detalle. Dado que estas asociaciones son redundantes, sólo se implementaría la asociación calificada; la otra se trataría como una restricción aplicada a sus componentes en tiempo de ejecución.

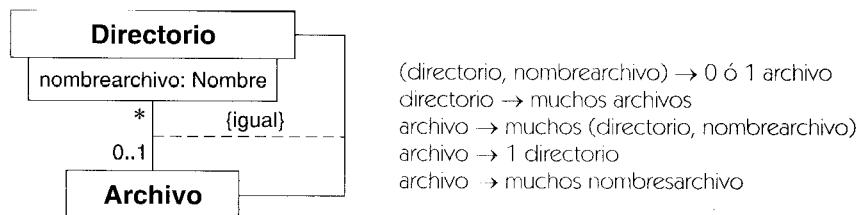


Figura 13.27 Un archivo con múltiples nombres en un directorio

Una restricción similar es que todo archivo puede aparecer en múltiples directorios pero siempre tiene el mismo nombre dondequiera que aparezca. Otros archivos podrán tener el mismo nombre pero tienen que estar en otro directorio. Esto se puede modelar haciendo que **nombearchivo** sea un atributo de **Archivo** pero restringiendo el atributo de clase y el calificador para que sean iguales (Figura 13.28). Este patrón aparece con frecuencia como índice de

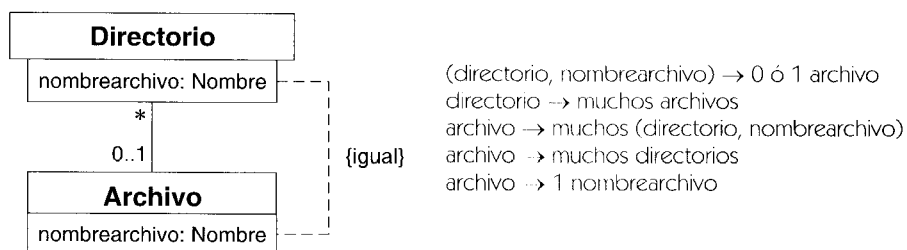


Figura 13.28 Un archivo con el mismo nombre en todos los directorios

búsqueda, aun cuando en un índice general la multiplicidad del destino calificado sería muchos. Por tanto, esta situación tiene más contenido sintáctico que un índice, que es un dispositivo de implementación.

Un tercer caso permitiría que un archivo apareciera en múltiples directorios con diferentes nombres, pero el archivo sólo podría aparecer una sola vez en cada directorio individual. Esto se podría modelar con una asociación calificada y una clase de asociación redundantes que compartieran el mismo atributo **nombreambre** (Figura 13.29).

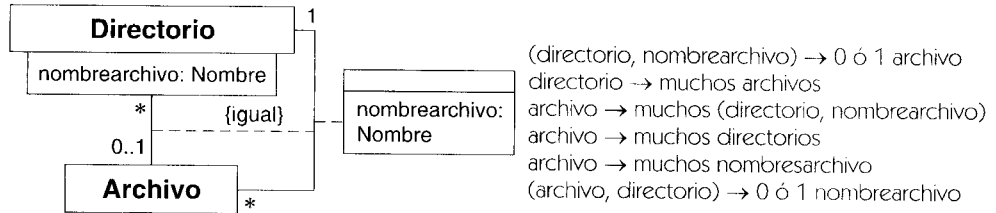


Figura 13.29 Un archivo con un nombre como máximo en cualquier directorio

Se han mostrado estos ejemplos con relaciones redundantes para ilustrar la naturaleza de las restricciones. Sin embargo, en la práctica suele resultar satisfactorio enunciar en forma de texto la restricción, mostrando gráficamente la asociación calificada.

calle¹

Partición de los grafos de actividades que se emplea para organizar las responsabilidades de las actividades. Las calles no tienen un significado fijo, pero suelen corresponder a las unidades organizativas en los modelos de negocios.

Véase también grafo de actividades.

Semántica

Los estados de actividad dentro de un grafo de actividades se pueden organizar en particiones llamadas calles como consecuencia de su notación. Las calles son agrupamientos de estados para organizar los grafos de actividades. Cada calle representa alguna partición significativa de las responsabilidades de los estados —por ejemplo, la organización de negocios responsable de un paso en un flujo de trabajo—. Se pueden utilizar en la forma que prefiera el creador del modelo. Las calles, si existen, reparten entre ellas los estados del grafo de actividades.

Cada calle posee un nombre distinto del de las otras calles. No posee ninguna semántica adicional dentro de UML pero puede tener algunas implicaciones propias del mundo real.

¹ Se alude al término empleado en natación, *swimlane*. N. del T.

Notación

Se puede dividir visualmente el diagrama de actividades en calles, cada una de las cuales está separada de las calles contiguas mediante una línea vertical continua (Figura 13.30). Cada calle representa alguna responsabilidad de alto nivel para una parte de la actividad global, que puede ser implementada eventualmente por uno o más objetos. El orden relativo de calles no posee significación semántica pero puede indicar alguna afinidad en el mundo real. Cada estado de actividad se asigna a un calle y se sitúa visualmente en su interior. Las transiciones pueden cruzar las calles; la ruta de una transición no tiene significado especial.

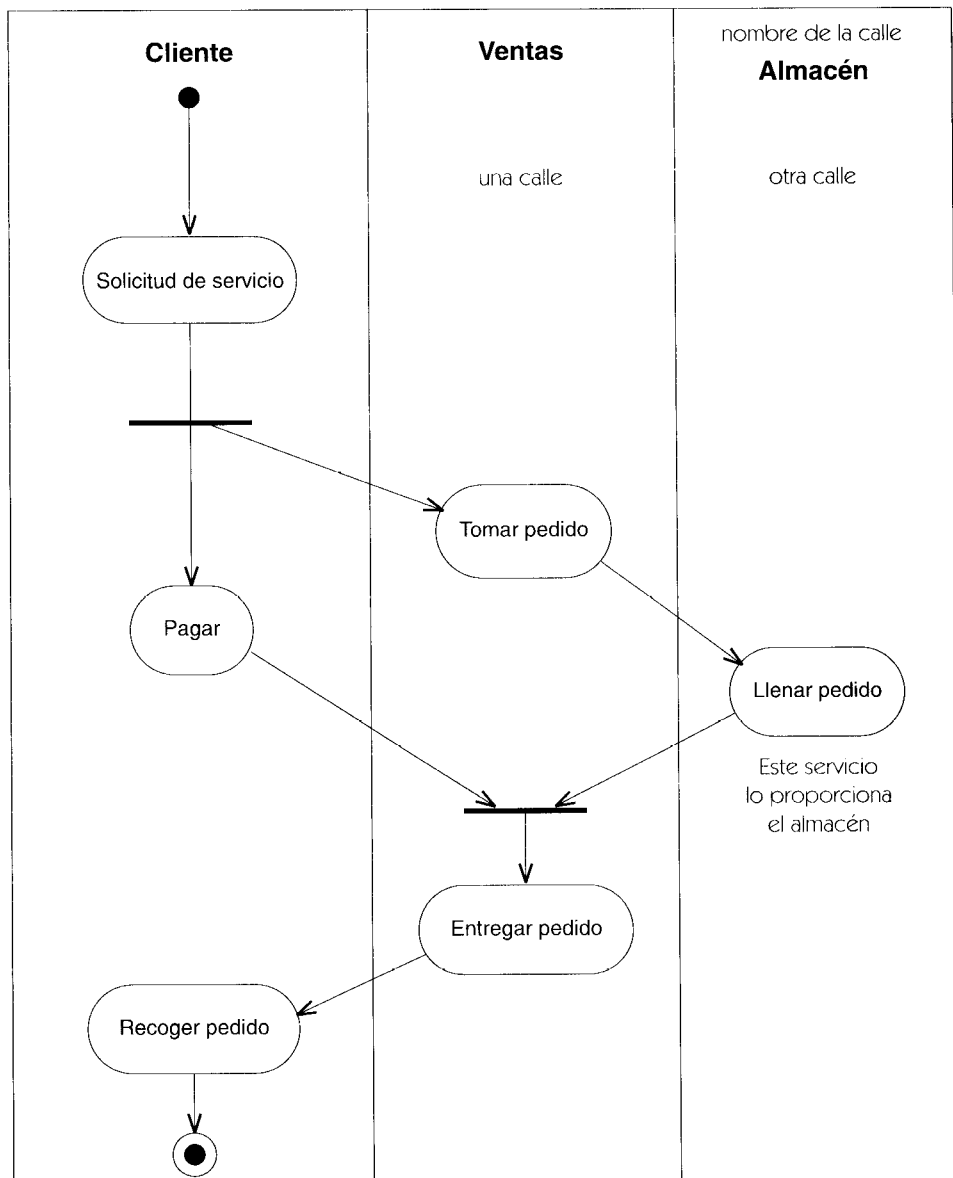


Figura 13.30 Calles de un diagrama de actividades

Como las calles no son otra cosa que particiones en categorías arbitrarias, pueden indicarse por otros medios si no resulta práctico agruparlas en regiones. Entre las posibilidades se cuenta el uso del color o utilizar simplemente valores etiquetados para mostrar la partición.

capa

Dícese de un patrón de arquitectura propio de los paquetes agrupados de un modelo, que poseen todos el mismo nivel de abstracción. Cada capa representa un mundo virtual en un determinado nivel de realidad.

característica

Una propiedad tal como una operación o atributo, que se encapsula como parte de una lista dentro de un clasificador, tal como una interfaz, una clase, o un tipo de dato.

característica de comportamiento

Elemento del modelo que expresa comportamiento dinámico, como una operación o método, y que puede ser una parte de un clasificador. La declaración de que un clasificador trata una señal, es también una característica de comportamiento.

Elementos estándar

create, destroy, leaf.

característica estructural

Característica estática de un elemento del modelo, tal como un atributo o una operación.

cardinalidad

Número de elementos de un conjunto. Es un número específico. Contrasta con multiplicidad, que es el rango de posibles cardinalidades que puede tener un conjunto.

Discusión

Obsérvese que muchos autores emplean el término *cardinalidad* de forma incorrecta queriendo hacer referencia a la multiplicidad, aunque el término cardinalidad tiene una definición matemática desde hace mucho tiempo como número, no como rango de números. Ésta es la definición que utilizamos aquí.

caso de uso

Especificación de las secuencias de acciones, incluyendo secuencias variantes y secuencias de error, que pueden ser efectuadas por un sistema, subsistema o clase por interacción con autores externos.

Véase también actor, clasificador.

Semántica

Un caso de uso es una unidad coherente de funcionalidad que proporciona un clasificador (un sistema, subsistema o clase) tal como lo manifiestan las secuencias de mensajes que se intercambian entre el sistema y uno o más usuarios externos (que se representan como actores), junto con acciones que realiza el sistema.

El propósito de un caso de uso es definir un cierto comportamiento de un clasificador (incluyendo un subsistema o todo el sistema) sin revelar la estructura interna del clasificador. Cada caso de uso especifica un servicio que proporciona el clasificador a sus usuarios, esto es, una forma específica de utilizar el clasificador que es visible desde el exterior. Describe una secuencia completa que es iniciada por un usuario (modelado por un actor) en términos de interacción entre los usuarios y el clasificador, así como las respuestas ofrecidas por el clasificador. La interacción sólo incluye las comunicaciones habidas entre el sistema y los actores. El comportamiento o implementación internos se ocultan. El conjunto completo de casos de uso de un clasificador o de un sistema particiona y abarca su comportamiento. Cada caso de uso representa un trozo cuantificado y significativo de funcionalidad que está disponible para los usuarios. Observe que entre los usuarios se cuentan los *seres* humanos así como los computadores y otros objetos. Un actor es la idealización del propósito de un usuario, no una representación de un usuario físico. Un usuario físico puede corresponderse con muchos actores y un actor puede representar el mismo aspecto de múltiples usuarios físicos.

Véase actor.

Los casos de uso incluyen el comportamiento normal y habitual que se tiene como respuesta a una solicitud del usuario, así como posibles variantes de la secuencia normal, tales como secuencias alternativas, comportamiento frente a excepciones y manejo de errores. El objetivo es describir un trozo de funcionalidad coherente en todas sus variaciones, incluyendo las condiciones de error. El conjunto completo de casos de uso de un clasificador especifica todas las formas distintas que hay de utilizar ese clasificador. Los casos de uso se pueden agrupar en paquetes por comodidad.

Un caso de uso es un descriptor; describe un comportamiento potencial. La ejecución de un caso de uso es una instancia de caso de uso. El comportamiento de un caso de uso se puede especificar mediante una máquina de estados asociada a él o bien por un código en forma de texto (que es equivalente a una máquina de estados). También se puede describir mediante una descripción informal en forma de texto. El comportamiento se puede ilustrar, pero no especificar formalmente, mediante un conjunto de escenarios. Esto puede resultar suficiente en las primeras fases del desarrollo.

Una instancia de caso de uso es una ejecución de un caso de uso, que será iniciada por un mensaje proveniente de una instancia de actor. Como respuesta al mensaje, la instancia de caso

de uso ejecuta una secuencia de acciones especificadas por el caso de uso, tales como enviar mensajes a instancias de actor, no necesariamente destinadas sólo al actor iniciador. Las instancias de actor pueden enviar mensajes a la instancia de caso de uso y la interacción prosigue hasta que la instancia ha respondido a todas las entradas. Cuando ya no espera más entradas, concluye.

Un caso de uso es una especificación del comportamiento de un sistema (u otro clasificador) como un todo en sus interacciones con actores exteriores. Las interacciones internas entre objetos internos del sistema que implementa el comportamiento se describen mediante una colaboración que realiza un caso de uso.

Estructura

Un caso de uso puede tener características y relaciones.

Características. Un caso de uso es un clasificador y por tanto tiene atributos y operaciones. Los atributos se emplean para representar el estado del caso de uso —esto es, el progreso de su ejecución—. Una operación representa un bloque de trabajo que puede realizar el caso de uso. No se puede invocar directamente desde el exterior, pero se puede emplear para describir el efecto del caso de uso sobre el sistema. La ejecución de una operación se puede asociar a la recepción de un mensaje procedente de un actor. Las operaciones actúan sobre los atributos del caso de uso e indirectamente sobre el sistema o clase al que esté asociado el caso de uso.

Asociaciones con actores. Una asociación entre un actor y un caso de uso indica que esa instancia de actor se comunica con la instancia de sistema o de clasificador para lograr algún resultado que sea de interés para el actor. Los actores modelan usuarios externos del clasificador. De este modo, si el clasificador es un sistema entonces sus actores son los usuarios externos del sistema. Los actores de subsistemas de nivel inferior pueden ser otras clases pertenecientes al sistema global.

Un actor se puede comunicar con varios casos de uso —esto es, el actor puede solicitar varios servicios diferentes del sistema— y un caso de uso puede comunicarse con uno o más actores cuando les proporciona sus servicios. Observe que dos casos de uso que especifiquen un mismo sistema no pueden comunicarse entre sí, porque cada uno de ellos describe individualmente un uso completo del sistema. Pueden interactuar indirectamente a través de actores compartidos.

La interacción entre actores y casos de uso se puede definir mediante interfaces. Una interfaz define las operaciones que puede admitir o utilizar un actor o caso de uso. Las distintas interfaces que ofrecen un mismo caso de uso no tienen necesidad de ser disjuntas.

Los casos de uso están relacionados con otros casos de uso mediante relaciones de generalización, extensión e inclusión.

Generalización. Una relación de generalización relaciona un caso de uso especializado con otro caso de uso más general. El hijo hereda los atributos, operaciones y secuencias de comportamiento del padre y puede agregar atributos y operaciones propios. El caso de uso hijo añade comportamiento al caso de uso padre insertando secuencias de acción adicionales en la secuencia del padre en puntos arbitrarios. También puede modificar algunas operaciones y secuencias heredadas, pero esto tiene que hacerse con igual cuidado que en una redefinición

para que la intención del padre se mantenga. Todas las relaciones de inclusión o extensión con el caso de uso hijo modifican también efectivamente el comportamiento heredado del caso de uso padre.

Extensión. Una relación de extensión es una especie de dependencia. El caso de uso cliente añade un comportamiento incremental al caso de uso base mediante la inserción de secuencias de acción adicionales a la secuencia base. El caso de uso cliente contiene uno o más segmentos separados de secuencia de comportamiento. La relación de extensión contiene una lista de nombres de puntos del caso de uso base, en número igual al número de segmentos que haya en el caso de uso cliente. Un punto de extensión representa una localización o conjunto de localizaciones dentro del caso de uso base en los cuales se podría insertar la extensión. Una relación de extensión también puede tener asociada una condición, que puede hacer uso de atributos del caso de uso padre. Cuando una instancia del caso de uso padre llega a una localización a la que hace referencia un punto de extensión de la relación de extensión, se evalúa la condición; si la condición es verdadera, entonces se ejecuta el correspondiente segmento de comportamiento del caso de uso hijo. Si no hay condición, se estima que es siempre cierta. Si la relación de extensión posee más de un punto de extensión, entonces la condición se evalúa únicamente en el primer punto de extensión antes de ejecutar el primer segmento.

Las relaciones de extensión no crean un nuevo caso de uso instanciable. Lo que hacen es añadir implícitamente comportamiento al caso de uso base original. El caso de uso base incluye implícitamente el comportamiento extendido. El caso de uso base original no extendido no está disponible en su forma no modificada. En otras palabras, si se extiende un caso de uso entonces no se puede instanciar explícitamente el caso de uso base sin la posibilidad de que haya extensiones. Un caso de uso puede tener múltiples extensiones que se aplicarán todas al mismo caso de uso base y se pueden insertar en una instancia de caso de uso si se satisfacen sus distintas condiciones. Por otra parte, un caso de uso de extensión puede extender varios casos de uso base (o el mismo en diferentes puntos de extensión), cada cual en su propio punto de extensión (o lista de puntos de extensión). Si hay varias extensiones en el mismo punto de extensión, su orden relativo de ejecución no es determinista.

Observe que el caso de uso de extensión no debe instanciarse; hay que instanciar el caso de uso base para obtener el comportamiento combinado de base más extensiones. El caso de uso de extensión puede ser o no instanciable pero en todo caso no incluye el comportamiento del caso de uso base.

Inclusión. Una relación de inclusión denota la inclusión de la secuencia de comportamiento del caso de uso proveedor en la secuencia de interacción de un caso de uso cliente, bajo el control del caso de uso cliente en una localización que especifique el cliente en su descripción. Se trata de una dependencia, no de una generalización, porque el caso de uso proveedor no puede ser reemplazado en aquellos casos en que aparece el caso de uso cliente. El cliente puede acceder a los atributos del caso de uso base para obtener valores y comunicar resultados. La instancia de caso de uso está ejecutando el caso de uso cliente. Cuando llega al punto de inclusión, comienza a ejecutar el caso de uso del proveedor hasta que éste finaliza. Después sigue ejecutando el caso de uso cliente más allá de la localización de la inclusión. Los atributos del caso de uso proveedor no tienen valores que persistan entre ejecuciones.

Un caso de uso puede ser abstracto, lo cual significa que se puede instanciar directamente en una ejecución de extensión. Define un fragmento de comportamiento que se especializa o se incluye en casos de uso concretos o quizá sea una extensión de un caso de uso base. También puede ser concreto si puede instanciarse por sí mismo.

Comportamiento. La secuencia de comportamiento de un caso de uso se puede describir mediante una máquina de estados, un grafo de actividades o un código de texto escrito en algún lenguaje ejecutable. Las acciones de la máquina de estados o las sentencias del código pueden llamar a las operaciones internas del caso de uso para especificar los efectos de la ejecución. Las acciones también pueden indicar el envío de mensajes a actores.

Un caso de uso se puede describir informalmente empleando escenarios o un texto normal, pero estas descripciones son imprecisas y sólo están destinadas a su interpretación por parte de seres humanos.

Las acciones de un caso de uso se pueden especificar en términos de llamadas a operaciones del clasificador que describe el caso de uso. Una operación puede ser invocada por más de un caso de uso.

Realización. La realización de un caso de uso se puede especificar mediante un conjunto de colaboraciones. Una colaboración describe la implementación del caso de uso por parte de objetos del clasificador que describe ese caso de uso. Cada colaboración describe el contexto entre constituyentes del sistema dentro del cual se produce una o más secuencias de interacción. Las colaboraciones y sus interacciones definen la forma en que interactúan los objetos del sistema para lograr el comportamiento externo especificado en el caso de uso.

Un sistema se puede especificar con casos de uso en diferentes niveles de abstracción. Un caso de uso que especifica un sistema, por ejemplo, se puede refinar para formar un conjunto de casos de uso subordinados, cada uno de los cuales especifica un servicio de un subsistema. La funcionalidad especificada por el caso de uso de rango superior se puede seguir por completo hasta la funcionalidad de los casos de uso subordinados (de rango inferior). Un caso de uso de rango superior y un conjunto de casos de uso de rango inferior especifican el mismo comportamiento en dos niveles distintos de abstracción. Los casos de uso subordinados cooperan para proporcionar el significado del caso de uso de rango superior. La cooperación de los casos de uso subordinados queda especificada por las colaboraciones del caso de uso de rango superior y se pueden representar mediante diagramas de colaboración. Los actores de un caso de uso de rango superior aparecen como actores de los casos de uso subordinados. Además, los casos de uso subordinados son actores entre sí. Esta realización por capas da lugar a un conjunto anidado de casos de uso y colaboraciones que implementan todo el sistema.

Notación

Los casos de uso se representan mediante una elipse que contiene el nombre del caso de uso. Si es preciso mostrar atributos o operaciones del caso de uso, el caso de uso puede dibujarse como un rectángulo clasificador con la palabra reservada «**use case**». La Figura 13.31 muestra un diagrama de caso de uso.

Un punto de extensión es una entidad con nombre perteneciente a un caso de uso que describe localizaciones en las cuales se pueden insertar secuencias de otros casos de uso. Proporciona un nivel de indirección entre las extensiones y el texto de secuencia de comportamiento. Un punto de texto hace referencia a una localización o conjunto de localizaciones dentro de la secuencia de comportamiento del caso de uso. Se puede cambiar la referencia independientemente de las relaciones de extensión que hagan uso del punto de extensión. Cada punto de extensión debe poseer un nombre único dentro del caso de uso. Los puntos de

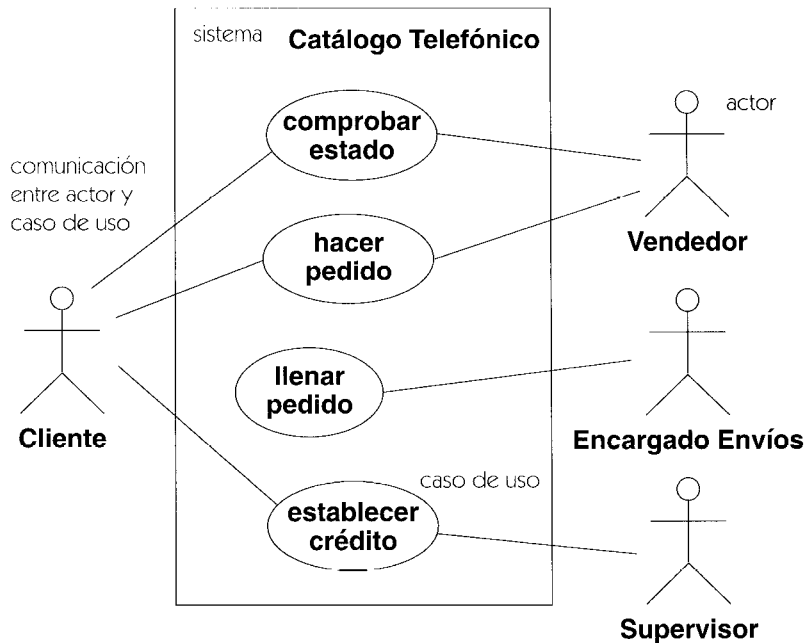


Figura 13.31 Casos de uso y actores

extensión se pueden enumerar en un compartimento del caso de uso, con el encabezado **puntos de extensión** (Figura 13.32).

Una relación de comunicación entre un caso de uso y un actor se muestra empleando un símbolo de asociación —una línea continua entre los símbolos del caso de uso y del actor—. Normalmente se puede omitir la palabra reservada «**communication**» porque éste es el único

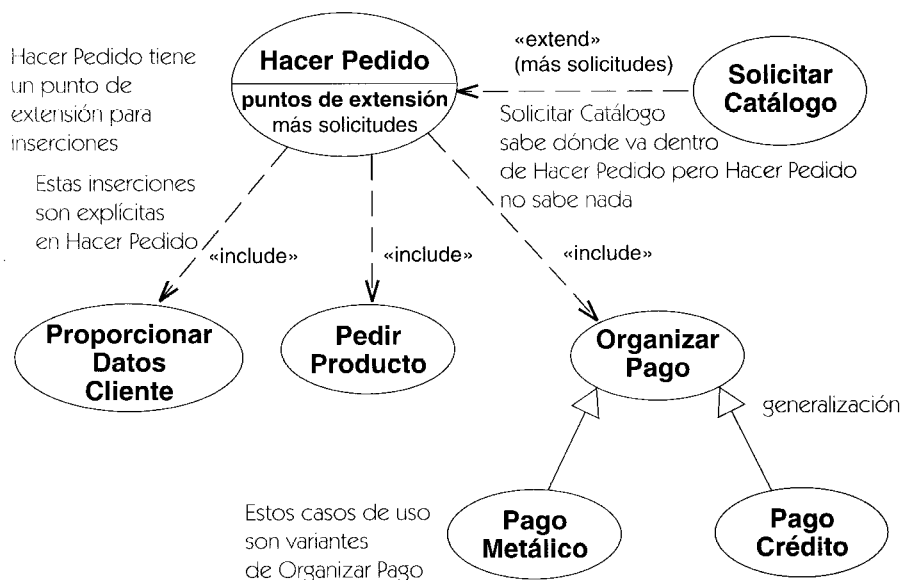


Figura 13.32 Relaciones de caso de uso

tipo de asociación que hay entre actores y casos de uso. Generalmente, no se ponen nombres ni nombres de rol en la línea, porque el actor y el caso de uso definen de modo único la relación.

Una relación de generalización se muestra mediante una flecha de generalización —una línea continua que va del caso de uso hijo al caso de uso padre, con una cabeza de flecha cerrada triangular en el caso de uso padre.

Una relación de extensión o de inclusión se muestran mediante una flecha de dependencia con la palabra reservada «**extend**» o «**include**» —una línea discontinua con cabeza de flecha hueca en el caso de uso cliente—. Las relaciones de extensión también tienen una lista de nombres de puntos de extensión (se pueden eliminar en el diagrama).

La Figura 13.32 muestra distintas clases de relaciones de casos de uso.

Especificación de comportamiento. La relación entre un caso de uso y sus secuencias externas de interacción suele representarse mediante un hipervínculo que llega a un diagrama de secuencias. El hipervínculo es invisible pero se puede recorrer mediante un editor. El comportamiento también se puede especificar mediante una máquina de estados o mediante un texto en algún lenguaje de programación asociado al caso de uso. Se puede emplear un texto en lenguaje natural como especificación informal.

Véase extensión para un ejemplo de secuencias de comportamiento.

La relación entre un caso de uso y su implementación se puede mostrar como una relación de realización que va del caso de uso a la colaboración. Pero dado que éstos suelen estar en modelos separados, suele representarse como un hipervínculo invisible. Se supone que una herramienta admitirá la posibilidad de “acercarse” a los casos de uso para contemplar sus escenarios y/o su implementación como una colaboración.

clase

Descriptor de un conjunto de objetos que comparten los mismos atributos, operaciones, métodos, relaciones y comportamiento. Una clase representa un concepto dentro del sistema que se está modelando. Dependiendo del tipo de modelo, el concepto puede ser del mundo real (en un modelo de análisis), o contener conceptos algorítmicos o de implementación en un computador (en un modelo de diseño). Un clasificador es una generalización del concepto de clase que incluye otros elementos similares a las clases como tipo de dato, actor y componente.

Semántica

Una clase es la descripción con nombre tanto de la estructura de datos como del comportamiento de un conjunto de objetos. Una clase se utiliza para declarar variables. Un objeto que es el valor de una variable debe tener una clase compatible con el tipo declarado para esa variable, o lo que es lo mismo, debe ser de la misma clase que la declarada o un descendiente de ella. Una clase también se utiliza para instanciar objetos; mediante operaciones de creación, se crean nuevas instancias de una clase.

Un objeto instanciado a partir de una clase es una instancia directa de la clase y una instancia indirecta de sus antecesores. El objeto contiene una ranura que mantiene un valor para

cada atributo; acepta todas las operaciones y señales de la clase, y puede aparecer en enlaces de asociaciones en los que participe la clase o un antecesor de la clase.

Algunas clases no pueden ser instanciadas directamente, ya que se emplean únicamente para describir estructuras compartidas por todos sus descendientes; a estas clases se las denomina clases abstractas. Una clase que puede ser instanciada directamente es una clase concreta.

Una clase también puede verse como un objeto global, siendo todos los atributos de la clase con alcance de clase los atributos de este objeto implícito. Estos atributos tienen un alcance global y cada uno de ellos tiene un único valor para todo el sistema. Una operación con alcance de clase se aplica a la clase, no al objeto, siendo las operaciones de creación las más comunes entre éstas.

En UML una clase es un tipo de clasificador. Los *clasificadores* son elementos similares a las clases, pero su expresión más clara es la *clase*.

Estructura

Una clase consta de un nombre y listas de operaciones, atributos y métodos. Una clase puede participar en una asociación, en una generalización, en dependencias y en relaciones de restricción. Se declara dentro de un espacio de nombres, como un paquete u otra clase, y tiene varias propiedades dentro de dicho espacio de nombres, como la multiplicidad o la visibilidad. Una clase tiene otras propiedades, como si es o no abstracta, o si es activa. Puede tener una máquina de estados que especifique su comportamiento reactivo, es decir, que especifique cómo responde a los eventos que recibe. Una clase puede declarar el conjunto de eventos (incluyendo excepciones) que es capaz de tratar. Puede proporcionar la realización del comportamiento especificado por cero o más interfaces o tipos proporcionando una implementación para dicho comportamiento. Una interfaz lista el conjunto de operaciones que ofrece una clase que promete realizar la interfaz.

Una clase contiene una lista de atributos y una lista de operaciones, cada una de las cuales forma un espacio de nombres dentro de la clase. Los atributos y las operaciones heredadas también aparecen dentro de sus respectivos espacios de nombres. Los espacios de nombres para atributos incluyen también los pseudoatributos, como los nombres de rol de las asociaciones que salen de la clase o los discriminadores de las relaciones de generalización en que participa la clase o alguno de sus antecesores. Un nombre debe declararse sólo una vez dentro de la clase y sus antecesores, pues de otro modo habría un conflicto y el modelo estaría mal formado. Los nombres de las operaciones se pueden repetir siempre que representen la misma operación, pues si no se produciría un conflicto de nombres.

Una clase también es un espacio de nombres y establece el alcance de las declaraciones de clasificadores anidados. Los clasificadores no son partes estructurales de las instancias de la clase. No existe relación entre los objetos de una clase y los objetos de clases anidadas. Una clase anidada es una declaración de una clase que puede ser utilizada por los métodos de la clase externa. Las clases declaradas dentro de otra clase son privadas a dicha clase, y por tanto no pueden ser accedidas desde fuera de la clase a menos que se las haga visibles explícitamente. No hay una notación para representar las declaraciones de clases anidadas, aunque es de esperar que en las herramientas pueda accederse a ellas mediante hipervínculos. Los nombres anidados deben ser referenciados utilizando la ruta del nombre.



Figura 13.33 Declaración básica de clase

Notación

Una clase se representa mediante un rectángulo con tres partes o compartimentos separados por líneas horizontales. La parte superior contiene el nombre de la clase y otras propiedades que se aplican a la totalidad de la clase. La parte central contiene una lista de atributos, mientras que la parte inferior contiene una lista de operaciones. Estas dos últimas partes se pueden suprimir en un símbolo de clase.

Utilización. Las clases se declaran en diagramas de clases y se utilizan en muchos otros diagramas. UML proporciona una notación gráfica para la declaración y uso de clases, y una notación textual para hacer referencia a las clases dentro de las descripciones de otros elementos. La declaración de una clase en un diagrama de clases define el contenido de la misma: atributos, operaciones y otras propiedades. Otros diagramas definen relaciones adicionales y otros elementos asociados a la clase.

La Figura 13.33 muestra una declaración básica de clase con atributos y operaciones.

La Figura 13.34 muestra la misma declaración de clase con detalles adicionales, sobre todo información sobre la naturaleza de su implementación, como visibilidad, operaciones fuente de creación de alcance de clase y operaciones dependientes de la implementación.

En la Figura 13.35 se suprime toda la información interna sobre la clase. No obstante, la información sigue existiendo en el modelo interno y podría mostrarse en otros diagramas.

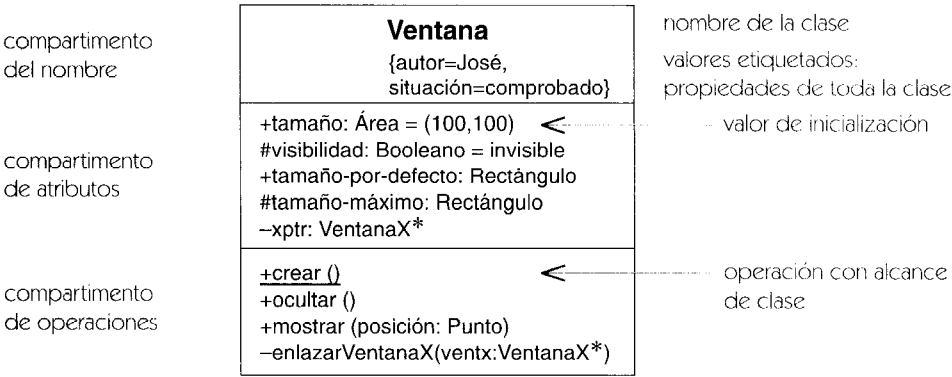


Figura 13.34 Declaración detallada de clase con visibilidad de características



Figura 13.35 Símbolo de clase con todos los detalles suprimidos

Opciones de representación

Supresión de compartimentos. Se puede suprimir cualquiera de los dos compartimentos ya citados (atributos y operaciones) o ambos, tal y como se muestra en la Figura 13.36. Cuando no se muestra un compartimento no se dibuja la línea que lo separa. En ese caso, no se puede inferir nada acerca de la presencia o ausencia de elementos en él. Téngase en cuenta que un compartimento vacío, es decir, uno con separador pero sin contenido, indica que no hay elementos en la lista correspondiente. Si se lleva a cabo algún tipo de filtrado, significa que no existe ningún elemento que satisfaga la condición de filtro. Por ejemplo, si sólo se desea mostrar las operaciones públicas, la presencia de un compartimento de operaciones vacío significa que no hay operaciones públicas, pero no se pueden sacar conclusiones acerca de las operaciones privadas.

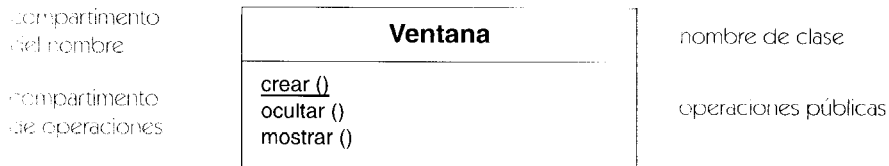
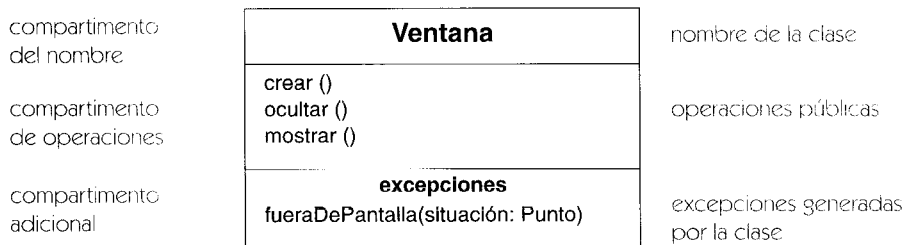


Figura 13.36 Declaración de clase con los atributos y operaciones no públicas suprimidas

Compartimentos adicionales. Se pueden añadir otros compartimentos para mostrar otras propiedades del modelo definidas por el usuario o predefinidas —por ejemplo, para mostrar reglas de negocio, variaciones, responsabilidades, tratamiento de señales, excepciones que genera, etcétera—. Un compartimento adicional se representa utilizando un nombre de compartimento en la parte superior que identifique de forma clara el contenido del mismo (Figura 13.37). Los compartimentos estándar (atributos y operaciones) no requieren nombre de compartimento, aunque pueden llevarlo para enfatizar o para mayor claridad cuando sólo hay un compartimento visible. La mayoría de los compartimentos son únicamente listas de cadenas, donde cada cadena representa una propiedad. Obsérvese que “cadena” puede ser también un icono o documentos empotrados, por ejemplo hojas de cálculo o gráficos. Es posible incluir formatos aún más complejos, pero UML no los especifica, ya que son responsabilidad del usuario y de las herramientas. Si la naturaleza del compartimento viene determinada por la forma de sus contenidos, se puede omitir el nombre del compartimento.

Véase uso de la tipografía, cadena.

Estereotipo. Un estereotipo se representa mediante un texto entre comillas (« ») encima del nombre de la clase, como se muestra en la Figura 13.38. Se puede utilizar también un icono en la esquina superior derecha del compartimento con nombre en lugar de la cadena de texto. Un símbolo de clase con un icono de estereotipo puede ser “comprimido” para mostrar sólo el icono de estereotipo, con el nombre de la clase dentro o debajo del icono (Figura 13.100). Se suprimen los demás contenidos de la clase.

**Figura 13.37** Compartimento adicional en la declaración de una clase

Véase estereotipo.

Reglas de estilo

- El nombre del estereotipo debe ir entre comillas, en letra normal y centrado sobre el nombre de la clase.
- El nombre de la clase debe ir en negrita, bien centrado o bien justificado a la izquierda.
- Los atributos y operaciones irán en letra normal justificados a la izquierda.
- El nombre de clases abstractas o la signatura de operaciones abstractas debe aparecer en cursiva.
- Muestre los compartimentos de atributos y operaciones cuando sea necesario (al menos una vez en todo el conjunto de diagramas), y suprímalos en otros contextos o referencias. Es útil definir un lugar “base” para una clase donde se muestre su descripción completa, mostrando la forma mínima en los demás sitios.

Discusión

El concepto de clase se aplica a un amplio espectro de diferentes utilizaciones en el modelado lógico y en la implementación, ya que incluye tanto el concepto de tipo como el concepto de clase de implementación. En UML, y bajo ciertos puntos de variación semántica, un objeto puede tener múltiples clases, así como ser capaz de cambiar de clase en tiempo de ejecución. Otras nociones más restrictivas de clase que aparecen en la mayoría de lenguajes de programación pueden asimilarse como tipos especiales de clases.

Elementos estándar

implementationClass, type.

**Figura 13.38** Clase con estereotipo

clase abstracta

Clase que no puede ser instanciada.

Véase abstracto/a.

Semántica

Una clase abstracta no puede tener instancias directas. Tiene que tener instancias indirectas a través de sus descendientes concretos.

Para una discusión sobre el tema, *véase abstracto/a.*

clase activa

Clase cuyas instancias son objetos activos.

Véase objeto activo para más detalles.

Semántica

Una clase activa es una clase cuyas instancias son objetos activos. Algunos estereotipos de clase activa son los procesos y los hilos.

Notación

Para indicar que una clase es activa se dibuja con un borde más grueso.

Ejemplo

La Figura 13.39 muestra un diagrama de clases con una clase activa y sus partes pasivas.

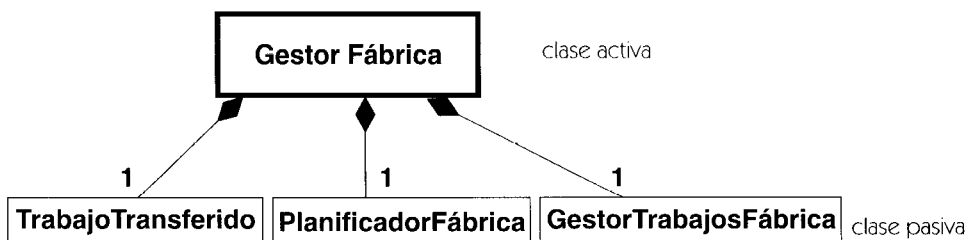


Figura 13.39 Clase activa y partes pasivas

La Figura 13.148 muestra una colaboración que contiene los objetos activos correspondientes a este modelo.

clase asociación

Una clase asociación es una asociación que también es una clase. Una clase asociación tiene propiedades tanto de las clases como de las asociaciones. Sus instancias son enlaces que tienen valores de los atributos a la vez que referencias a otros objetos. Aunque su notación esté formada por los dos símbolos, el de la asociación y el de la clase, es un único elemento del modelo.

Véase también asociación, clase.

Semántica

Una clase asociación tiene las propiedades de las asociaciones, pues conecta dos o más clases, y también tiene atributos y operaciones. Una clase asociación es útil cuando cada enlace debe tener sus propios valores para los atributos, operaciones propias, o sus propias referencias a objetos. Puede verse como una clase con una referencia de clase extra por cada extremo de la asociación, que es la forma obvia y normal de implementarlo. Cada instancia de una clase asociación tiene referencias a objetos, así como los valores para los atributos especificados por la parte de clase.

Una clase asociación C que conecta dos clases A y B no es lo mismo que una clase D con una asociación binaria a cada una de las clase A y B (véase la sección de discusión). Como todos los enlaces, un enlace de una clase asociación como C toma su identidad de las referencias a objetos que contiene. Los valores de los atributos no intervienen para proporcionarle su identidad, por lo que nunca dos enlaces de C podrán tener el mismo par de objetos (a, b) aunque los valores de sus atributos sean diferentes, pues tendrían entonces la misma identidad. Esto es, dado un atributo E, no está permitido que dos instancias de C tengan los valores (a, b, e1) y (a, b, e2) porque comparten la misma identidad (a, b). Sin embargo, los objetos tienen identidad inherente, por lo que dos objetos pueden tener idénticos valores de sus atributos o enlaces a los mismos objetos. En otras palabras, una asociación, incluyendo una clase asociación como C, es un conjunto de tuplas y no tiene duplicados entre sus referencias a objetos; mientras que una relación implícita como D es más como una bolsa, que puede tener duplicados. Para más detalles, véase la sección de discusión.

Las clases asociación pueden tener operaciones que modifiquen los atributos del enlace o que añadan o eliminen enlaces al propio enlace. Como la clase asociación es una clase, puede participar en asociaciones como tal clase.

Una clase asociación no puede tenerse a sí misma como una de sus clases participantes (aunque sin duda se podría encontrar un significado para una estructura recursiva de este tipo).

Notación

Una clase asociación se representa mediante un símbolo de clase (un rectángulo) unido por una línea discontinua a la ruta de una asociación (véase Figura 13.40). El nombre del símbolo de la

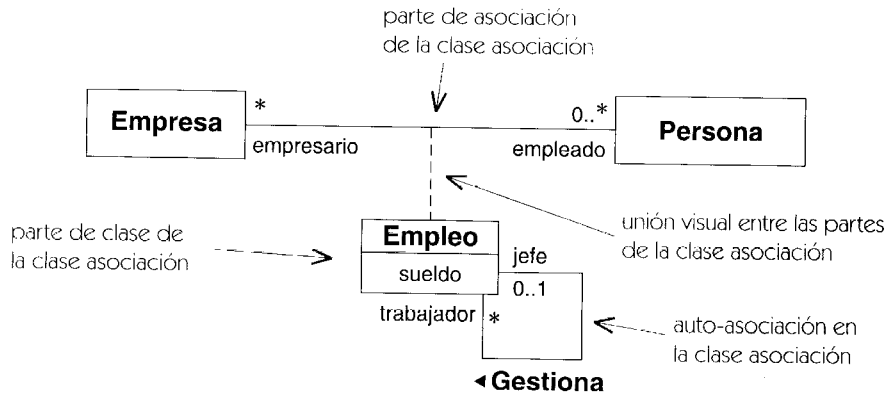


Figura 13.40 Clase asociación

clase y la cadena del nombre que acompaña a la ruta de la asociación son redundantes. La ruta de asociación puede tener los habituales adornos en los extremos. El símbolo de la clase puede tener atributos y operaciones, y participar en otras asociaciones como clase. No hay adornos en la línea discontinua, ya que no es una relación, sino simplemente parte del símbolo global de la clase asociación.

Reglas de estilo

El punto de unión no debería estar tan cerca de uno de los extremos de la asociación que pudiera dar lugar a pensar que está unido al final de la ruta o a alguno de los adornos de los roles.

Obsérvese que la ruta de asociación y la clase asociación son un único elemento del modelo, por lo que tienen un único nombre. El nombre puede aparecer sobre la ruta, en el nombre de la clase o en ambos. Si la clase asociación sólo tiene atributos pero no operaciones u otras asociaciones, el nombre puede mostrarse en la ruta de la asociación y omitirse en el símbolo de la clase asociación para enfatizar su "naturaleza de asociación", mientras que si tiene operaciones u otras asociaciones, es mejor no mostrar el nombre en la ruta y situarlo en el rectángulo de la clase para poner de relieve su "naturaleza de clase". En ninguno de los dos casos cambia la semántica.

Discusión

La Figura 13.40 muestra una clase asociación que representa el empleo. La asociación de empleo entre una empresa y una persona es muchos a muchos. Una persona puede tener varios

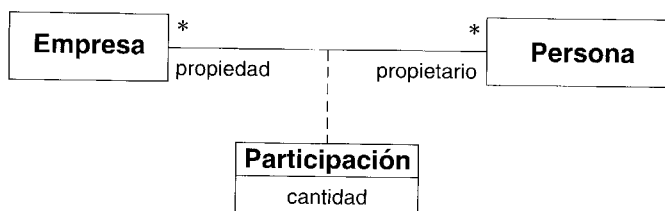


Figura 13.41 Clase asociación con atributo

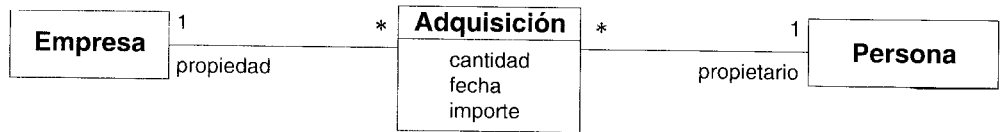


Figura 13.42 Asociación materializada

trabajos, pero sólo uno en una empresa determinada. El sueldo no es un atributo ni de la empresa ni de la persona puesto que la relación es muchos a muchos, por lo que debe ser un atributo de la relación.

La relación trabajador-jefe no es sólo una relación entre dos personas. Es una relación entre una persona en un trabajo y otra persona en otro trabajo. Es una asociación (**Dirige**) entre la clase asociación y ella misma.

El siguiente ejemplo muestra la diferencia entre una clase asociación y una relación materializada modelada como clase. En la Figura 13.41, la propiedad de la empresa se modela como una relación entre **Persona** y **Empresa**. El atributo **cantidad** de la clase asociación representa el número de acciones de la empresa que tiene una persona. La relación se modela como una clase asociación porque sólo debería haber una entrada para cada par **Persona-Empresa**.

Para modelar la adquisición de acciones, como se muestra en la Figura 13.42, no utilizamos una clase asociación, ya que puede haber más de una compra de acciones de una empresa por parte de la misma persona. Las compras deben distinguirse porque cada una es distinta y tiene su propia fecha y precio, además obviamente de la cantidad. La relación debe pues materializarse, esto es, dividirse en distintos objetos con identidad propia. La forma correcta de modelarlo es utilizar una clase ordinaria, ya que cada compra tiene su propia identidad, independiente de las clases **Persona** y **Empresa** que relaciona. Ésta es la forma de modelar una relación que es una bolsa en lugar de un conjunto.

clase compuesta

Una clase que está relacionada con una o más clases por una relación de composición.

Véase composición.

clase de implementación

En inglés, *Implementation Class*. Es un estereotipo de una clase que proporciona una implementación física, incluyendo los atributos, asociaciones a otras clases y métodos para las operaciones. La clase de implementación está orientada a lenguajes orientados a objetos tradicionales con clasificación estática simple. Un objeto de un sistema de estas características debe tener exactamente una clase de implementación como clase directa. Compárese con tipo, un estereotipo de una clase que permite clasificación múltiple. En un lenguaje convencional, tal como Java, un objeto puede tener una sola clase de implementación y muchos tipos. La clase de implementación debe ser consistente con los tipos.

Véase tipo, donde se comparan los tipos con las clases de implementación.

clase directa

La clase que describe lo más completamente posible un objeto.

Véase también clase, generalización, herencia, clasificación múltiple, herencia múltiple.

Semántica

Un objeto puede ser una instancia de muchas clases —si es una instancia de una clase, entonces también es una instancia de los antecesores de la clase. La clase directa es la descripción más específica de un objeto, la que lo describe lo más completamente posible. Un objeto es una instancia directa de su clase directa y una instancia indirecta de los antecesores de la clase directa. Un objeto no es una instancia de ningún descendiente de la clase directa (por definición).

Si la clasificación múltiple está permitida en el sistema, ninguna clase directa puede describir totalmente a un objeto. El objeto puede ser la instancia directa combinada de más de una clase. Un objeto es una instancia directa de cada clase que contenga parte de su descripción, con tal que ningún descendiente describa también al objeto.

En otras palabras, ninguna de las clases directas de un objeto tiene una relación ancestral con las otras.

Si se instancia una clase para producir un objeto, el objeto es una instancia directa de la clase.

clase-en-un-estado

Una clase junto con un estado en que los objetos de la clase pueden estar.

Véase también grafo de actividades.

Semántica

Una clase con una máquina de estados tiene muchos estados, cada uno de los cuales caracteriza el comportamiento, valores y restricciones de las instancias que se encuentran en dicho estado. En algunos casos, ciertos atributos, asociaciones u operaciones sólo son válidos cuando un objeto está en un determinado estado; en otros casos, un argumento de una operación debe ser un objeto en un determinado estado. A menudo estas distinciones son simplemente parte de los modelos de comportamiento, pero a veces es útil modelarlas directamente en vistas estáticas o en vistas de interacción.

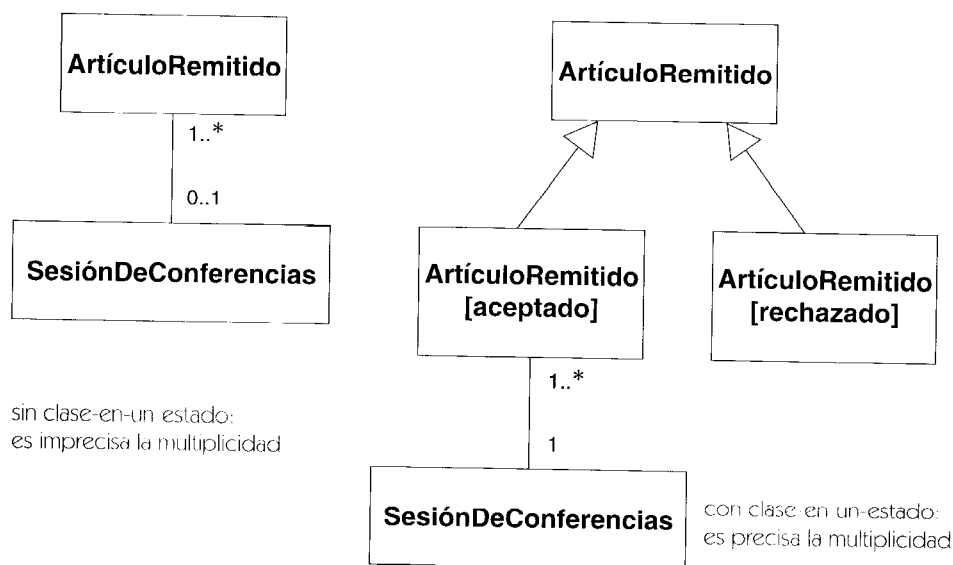


Figura 13.43 Clase-en-un-estado

Una clase en un estado es una clase, junto con uno de los estados válidos en los que los objetos de la clase pueden estar. Si la clase tiene subestados concurrentes, la especificación de estado puede ser un conjunto de subestados en los que la clase puede estar simultáneamente. Una clase en un estado puede utilizarse como clasificador; se comporta como una subclase de la propia clase; puede utilizarse como la clase de una variable o de un parámetro; y puede participar en asociaciones que sólo son válidas para objetos que estén en el estado dado. Considere la asociación **Asignación** de la Figura 13.43 entre las clases **SesiónDeConferencias** y **ArtículoRemitido**. Esta asociación es válida para un **ArtículoRemitido** en el estado **aceptado** (multiplicidad destino igual a uno) pero no en el estado **rechazado**. Para cualquier **ArtículoRemitido**, la multiplicidad destino es cero o uno, puesto que la clase incluye artículos tanto aceptados como rechazados. Sin embargo, si se modela la asociación entre **ArtículoRemitido** en el estado **aceptado** y **SesiónDeConferencias**, la multiplicidad destino tiene que ser exactamente uno.

Los elementos de una clase-en-un-estado son también útiles para mostrar los valores de entrada y salida de operaciones en grafos de actividades.

Notación

Una clase-en-un-estado se representa mediante un símbolo de clase en el cual el nombre de la clase va seguido del nombre del estado entre corchetes (**NombreClase[nombreEstado]**). Los corchetes también pueden contener una lista de nombres de estados concurrentes separados por comas para indicar que un objeto está en varios estados.

Discusión

Las clases-en-un-estado y la clasificación dinámica son dos formas de dar cumplimiento al mismo objetivo de permitir a las clases cambios en la estructura de los objetos durante su vida. Se utilizará el más conveniente de los dos dependiendo del entorno de implementación.

clase unitaria

Clase que posee (por declaración) una única instancia. Una clase unitaria es la forma de representar el conocimiento global en una aplicación, manteniéndolo en un marco de trabajo orientado a objetos.

Semántica

Toda aplicación debe poseer al menos una clase unitaria (frecuentemente, de forma implícita) para establecer el contexto de la aplicación. Con frecuencia, la clase unitaria equivale a la aplicación en sí y está implementada por la pila de control y el espacio de direcciones del computador.

Notación

Las clases unitarias se representan mediante un símbolo de clase con un pequeño “1” en la esquina superior derecha (Figura 13.44). Este valor representa la multiplicidad de la clase dentro del sistema.



Figura 13.44 Clase unitaria

clasificación dinámica

Una variación semántica de la generalización en la que un objeto puede cambiar el tipo o el rol. Contrastar con: clasificación estática.

Véase también clasificación múltiple.

Semántica

En muchos lenguajes de programación, un objeto no puede cambiar la clase de la cual es instanciado. Esta restricción estática de la clasificación simplifica la implementación y la optimización de compiladores, pero no es una necesidad lógica. Por ejemplo, asumiendo clasificación estática, un objeto instanciado como un círculo debe seguir siendo un círculo; no se puede cambiar su tamaño en la dimensión X, por ejemplo. Suponiendo que hay clasificación dinámica, un círculo del que se cambia dimensión se convierte en una elipse. Esto no se considera un problema.

Cualquier asunción puede utilizarse en un modelo de UML. Éste es un ejemplo de un punto de variación semántica. Sorprendentemente, la elección afecta poco al modelo, aunque

las diferencias son importantes para la ejecución. En ambos casos se definen las mismas clases, pero sus operaciones pueden diferir.

clasificación estática

Variante semántica de la generalización en la cual un objeto no puede cambiar de tipo o no puede cambiar de rol. La selección de la clase estática frente a la clasificación dinámica es un punto de variación semántica.

clasificación múltiple

Se trata de una variación semántica de generalización en la cual un objeto puede pertenecer directamente a más de una clase.

Semántica

Éste es el punto de variación semántica en el cual un objeto puede ser instancia directa de más de una clase. Cuando se usa con clasificación dinámica, los objetos pueden adquirir y perder clases durante el tiempo de ejecución. Éste permite utilizar clases para representar roles temporales que puede desempeñar un objeto.

Aun cuando la clasificación múltiple se adapta bien a la lógica y al discurso habitual, complica la implementación de un lenguaje de programación y no es admitida por los lenguajes populares de programación.

clasificación simple

Régimen de ejecución en que cada objeto posee exactamente una clase directa. Es el modelo de ejecución de la mayoría de los lenguajes de programación orientados a objetos. Admitir la clasificación simple o la clasificación múltiple es un punto de variación semántica.

clasificador

Elemento del modelo que describe características estructurales y de comportamiento. Las clases, actores, componentes, tipos de datos, interfaces, nodos, señales, subsistemas y casos de uso son tipos de clasificadores. El tipo más general de clasificador es la clase. Otros tipos pueden tener un cierto parecido intuitivo con las clases, con ciertas restricciones sobre el contenido y la utilización, aunque cada tipo de clasificador está representado por su propia clase en el metamodelo. En general, la mayoría de las propiedades de las clases se aplican a los clasificadores, con ciertas restricciones según el tipo de clasificador.

Véase también elemento generalizable, vista estática.

Elementos estándar

enumeration, location, metaclass, persistence, powertype, process, semantics, stereotype, thread, utility.

cliente

Elemento que solicita un servicio a otro elemento. El término se utiliza para describir un rol dentro de una dependencia. En la notación, el cliente aparece en el origen de una flecha de dependencia. Antónimo: proveedor.

Véase dependencia.

colaboración

Descripción de una disposición general de objetos y enlaces que interactúan dentro de un contexto para implementar un comportamiento, como un caso de uso o una operación. Una colaboración tiene una parte estática y una parte dinámica. La parte estática describe los roles que pueden desempeñar los objetos y enlaces en una instancia de la colaboración. La parte dinámica está formada por una o más interacciones dinámicas que muestran flujos de mensajes en la colaboración a través del tiempo para realizar cómputos.

Véase también mensaje, interacción, rol de asociación, rol de clasificador.

Semántica

El comportamiento lo implementan grupos de objetos que intercambian mensajes en un contexto determinado para llevar a cabo un cierto propósito. Para comprender el mecanismo utilizado en un diseño, es importante fijarse en los objetos y mensajes implicados en la realización del propósito o del conjunto de propósitos relacionados, proyectados desde el sistema más amplio dentro del cual realizan también otros propósitos. Una disposición de objetos y enlaces que trabajan juntos para llevar a cabo un objetivo se denomina colaboración; una secuencia de mensajes que implementan un comportamiento dentro de una colaboración se denomina interacción. Una colaboración es una descripción de una "sociedad de objetos"; es un fragmento de un modelo más amplio y más completo, dentro del cual la colaboración es un propósito.

Por ejemplo, una venta representa una disposición de objetos que tienen ciertas relaciones entre ellos dentro de la transacción. Las relaciones no tienen sentido fuera de la transacción. Los participantes en una venta son el comprador, el vendedor y un intermediario. Para realizar una interacción específica, como la venta de una casa, los participantes intercambian una determinada secuencia de mensajes, como la presentación de una oferta o la firma de un contrato.

Los flujos de mensajes de una colaboración pueden especificarse de forma opcional utilizando una máquina de estados que muestre las secuencias válidas de comportamiento. Los eventos de la máquina de estados representan los mensajes que los roles intercambian dentro de la colaboración.

Una colaboración está formada por roles. Un rol es una parte que desempeña un clasificador o una asociación dentro de la colaboración. Un rol es una ranura que puede contener una instancia de un clasificador o de una asociación. El mismo clasificador o la misma asociación pueden desempeñar diferentes roles, cada uno de los cuales podría ser ocupado por un objeto o enlace diferente. Por ejemplo, dentro de una transacción comercial, una parte puede ser el vendedor y otra el comprador, incluso aunque ambos sean empresas. El **comprador** y el **vendedor** son instancias de la clase **Empresa** dentro de la colaboración **Venta**. Los roles sólo tienen significado dentro de una colaboración; fuera de ella carecen de sentido. Además, en otras colaboraciones se pueden intercambiar los roles: un objeto puede ser **comprador** en una instancia de la colaboración y **vendedor** en otra. El mismo objeto puede desempeñar múltiples roles en colaboraciones diferentes. Contraste el restringido alcance de un rol con una asociación. Una asociación describe una relación con significado completo para una clase en todos los contextos, participe o no realmente un objeto en la asociación. Una colaboración define relaciones que se encuentran restringidas a un contexto y que por tanto no tienen sentido fuera del mismo.

Realización. Una colaboración realiza una operación o un caso de uso. Describe un contexto en el que se ejecuta la implementación de una operación o caso de uso —esto es, la disposición de objetos y enlaces existente en el momento de inicio de la ejecución, y las instancias que se crean y se destruyen durante la misma—. Se pueden especificar las secuencias de comportamiento en interacciones, representándolas como diagramas de secuencia o como diagramas de colaboración.

Una colaboración también puede realizar la implementación de una clase. Una colaboración para una clase es la unión de las colaboraciones para sus operaciones. Se pueden concebir diferentes colaboraciones para la misma clase, sistema o subsistema, de forma que cada colaboración muestre el subconjunto de atributos, operadores y objetos relacionados relevantes en una determinada vista de la entidad, como la implementación de una operación concreta.

Patrones. Una colaboración parametrizada representa una construcción de diseño que puede ser reutilizada en varios diseños. Generalmente los clasificadores base son parámetros. Una colaboración parametrizada de este tipo captura la estructura de un *patrón*.

Véase plantilla.

Un patrón de diseño se instancia proporcionando clasificadores reales para los parámetros de clasificadores base.

Cada instanciación produce una colaboración entre un conjunto específico de clasificadores del modelo. Un patrón puede ser ligado una o más veces con diferentes conjuntos de clasificadores dentro de un modelo, evitando así la necesidad de definir una colaboración para cada ocurrencia. Por ejemplo, un patrón de una vista del modelo define una relación general entre elementos del modelo, por lo que puede ligarse con muchas parejas de clases para representar parejas de vistas del modelo. Cada pareja de clases vista del modelo representará pues una ligadura del patrón. Una pareja como ésta podría ser una casa y una fotografía de la casa, otra pareja podría ser un stock y una gráfica que mostrara el precio actual del stock.

Obsérvese que un patrón implica también la existencia de unas directrices para su uso, ventajas y desventajas. Estas directrices pueden ponerse en notas o en documentos de texto independientes.